

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nguyễn Sỹ Tân

**XÂY DỰNG HỆ THỐNG CHỦ ĐỘNG TỐI ƯU HÓA
PHÂN BỐ TÀI NGUYÊN CHO HẠ TẦNG Đám Mây
OPENSTACK**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

HÀ NỘI - 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



NGUYỄN SỸ TÂN

**XÂY DỰNG HỆ THỐNG CHỦ ĐỘNG TỐI ƯU HÓA
PHÂN BỐ TÀI NGUYÊN CHO HẠ TẦNG Đám Mây
OPENSTACK**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

Cán bộ hướng dẫn: PGS.TS. Phạm Mạnh Linh

HÀ NỘI - 2026

Lời cảm ơn

Lời đầu tiên, em xin bày tỏ lòng biết ơn sâu sắc đến giảng viên hướng dẫn của em - PGS.TS. Phạm Mạnh Linh, người đã tận tình hướng dẫn, chỉ bảo và truyền đạt những kiến thức quý báu trong suốt quá trình em nghiên cứu khóa luận. Sự tận tâm và trách nhiệm của thầy là nguồn động viên lớn giúp em vượt qua những khó khăn và hoàn thành khóa luận này một cách tốt nhất.

Em cũng xin bày tỏ lòng biết ơn sâu sắc đến Khoa Công nghệ Thông tin - Trường Đại học Công nghệ, ĐHQGHN đã tạo điều kiện tốt nhất về cơ sở vật chất, trang thiết bị và môi trường học tập lý tưởng để em có thể nghiên cứu và thực hiện khóa luận này cũng như hướng dẫn, chỉ bảo em trong suốt thời gian em học tập và nghiên cứu tại trường.

Cuối cùng, em xin chân thành cảm ơn tất cả mọi người đã đồng hành và ủng hộ em trên con đường này. Chúc mọi người luôn may mắn, hạnh phúc và thành công trong mọi lĩnh vực của cuộc sống.

Hà Nội, ngày ... tháng ... năm 2026

Sinh viên

Nguyễn Sỹ Tân

Lời cam đoan

Em là Nguyễn Sỹ Tân, sinh viên lớp K67I-CS2. Em xin cam đoan rằng khóa luận tốt nghiệp với đề tài "Xây dựng hệ thống chủ động tối ưu hóa phân bổ tài nguyên cho hạ tầng đám mây OpenStack" là sản phẩm của công sức và nỗ lực do chính em thực hiện, dưới sự hướng dẫn của PGS.TS. Phạm Mạnh Linh. Các số liệu, kết quả và tài liệu trình bày trong khóa luận đều được thực hiện một cách độc lập và trung thực, chưa từng được công bố trong bất kỳ công trình nào khác, trừ những tài liệu tham khảo đã được trích dẫn rõ ràng trong danh mục tài liệu tham khảo.

Trong quá trình thực hiện khóa luận, em có sử dụng các công cụ Gen AI như ChatGPT, Claude để hỗ trợ lập trình, sửa lỗi. Các công cụ này chỉ được sử dụng với mục đích hỗ trợ kỹ thuật, không thay thế cho quá trình phân tích và nghiên cứu độc lập của em. Toàn bộ nội dung học thuật, ý tưởng thiết kế hệ thống và kết quả nghiên cứu đều do em tự thực hiện dưới sự hướng dẫn của giảng viên, không phụ thuộc hay mù quáng chấp nhận các nội dung do các công cụ tạo ra mà không hiểu rõ bản chất.

Em cam kết tuân thủ mọi quy định và hướng dẫn của Trường Đại học Công nghệ - DHQGHN, em xin chịu hoàn toàn trách nhiệm về tính chính xác và trung thực của nội dung khóa luận này.

Hà Nội, ngày ... tháng ... năm 2026

Sinh viên

Nguyễn Sỹ Tân

Tóm tắt

Tóm tắt: Trong bối cảnh phát triển mạnh mẽ của các nền tảng ảo hóa và điện toán đám mây, việc quản lý và tối ưu hóa phân bổ tài nguyên đóng vai trò quan trọng nhằm đảm bảo hiệu năng, tính ổn định và hiệu quả vận hành của hệ thống. Đặc biệt trong môi trường đám mây OpenStack, khi số lượng máy ảo (Virtual Machine - VM) và khối lượng công việc (workload) gia tăng nhanh chóng, tình trạng mất cân bằng tải giữa các máy chủ (host) chạy ảo hóa thường xuyên xảy ra, dẫn đến hiện tượng quá tải cục bộ tại một số host trong khi các host khác vẫn còn dư thừa tài nguyên, gây lãng phí và làm suy giảm hiệu năng chung của toàn hệ thống. Xuất phát từ thực tế đó, khóa luận này đề xuất một hệ thống hỗ trợ tối ưu phân bổ VM thông minh dựa trên phân tích dữ liệu và ứng dụng các mô hình trí tuệ nhân tạo mang tên OpenstackDRS. Hệ thống được thiết kế với các chức năng chính bao gồm: thu thập và tổng hợp dữ liệu tài nguyên từ hệ thống giám sát, xử lý và chuẩn hóa dữ liệu đầu vào, sử dụng mô hình dự báo chuỗi thời gian (cụ thể là Amazon Chronos) để dự đoán mức sử dụng tài nguyên của các máy chủ trong ngắn hạn, và cuối cùng là đánh giá trạng thái cân bằng của cụm nhằm phát hiện sớm các host có nguy cơ quá tải. Dựa trên các thông tin dự báo này, hệ thống đưa ra đề xuất phân bổ và di chuyển VM phù hợp nhằm tối ưu hóa cân bằng tải giữa các host, đồng thời hạn chế số lượng lần di chuyển không cần thiết để giảm chi phí vận hành và tránh ảnh hưởng đến hiệu năng hệ thống. Bên cạnh đó, các ràng buộc thực tế như affinity¹, anti-affinity² cũng được tích hợp trực tiếp vào quá trình ra quyết định, đảm bảo tính khả thi, an toàn và phù hợp với yêu cầu triển khai trong môi trường thực tế. Thực nghiệm trên 9 kịch bản cho thấy OpenstackDRS đạt tỷ lệ thành công 9/9, với tổng ΔIS tích lũy đạt +29,7% sau 9 lần migration. Hệ thống duy trì FP Rate 0% (0/8 migration không cần thiết), cho thấy khả năng đáp ứng tốt hiệu quả để có thể đưa vào vận hành.

Từ khóa: Điện toán đám mây, Phân bổ tài nguyên, Dự báo chuỗi thời gian.

¹Affinity là ràng buộc ưu tiên hoặc bắt buộc đặt các VM trên cùng một host vật lý nhằm giảm độ trễ giao tiếp hoặc đáp ứng yêu cầu triển khai.

²Anti-affinity là ràng buộc ưu tiên hoặc bắt buộc đặt các VM trên các host vật lý khác nhau nhằm tăng tính sẵn sàng, chịu lỗi hoặc tránh tranh chấp tài nguyên.

Mục lục

Lời cảm ơn	
Lời cam đoan	i
Tóm tắt	ii
Mục lục	iii
Danh sách hình vẽ	v
Danh sách bảng	vi
Danh mục các từ viết tắt	vii
Mở đầu	1
Chương 1 Cơ sở lý thuyết	5
1.1 Ảo hóa và điện toán đám mây	5
1.1.1 Ảo hóa	5
1.1.2 Điện toán đám mây	6
1.1.3 Đám mây OpenStack	8
1.2 Giám sát hệ thống	10
1.2.1 Prometheus	10
1.2.2 Grafana	11
1.3 Bài toán dự đoán chuỗi thời gian và Amazon-Chronos	13
1.3.1 Bài toán dự đoán trong hệ thống	13
1.3.2 Mô hình ngôn ngữ lớn và Amazon Chronos	14
1.4 Tổng kết chương	16
Chương 2 Phân tích và thiết kế hệ thống	17
2.1 Yêu cầu hệ thống	17

2.1.1	Yêu cầu chức năng	17
2.1.2	Yêu cầu phi chức năng	18
2.2	Kiến trúc tổng quan hệ thống	19
2.2.1	Mô hình kiến trúc	19
2.2.2	Luồng xử lý dữ liệu	20
2.3	Thiết kế chi tiết các thành phần	22
2.3.1	Collector Service	22
2.3.2	Analytics Service	23
2.3.3	Scoring Service	24
2.3.4	Engine Service	26
2.3.5	API	29
2.3.6	Dashboard	30
2.3.7	Database	32
2.4	Mô hình dự đoán chuỗi thời gian	33
2.4.1	So sánh và lựa chọn mô hình dự đoán	33
2.4.2	Thiết kế fine-tuning mô hình Chronos	34
2.5	Tổng kết chương	41
Chương 3	Thực nghiệm và đánh giá	42
3.1	Môi trường thực nghiệm	42
3.2	Mô tả cách thức thực nghiệm	43
3.3	Các kịch bản thực nghiệm	44
3.4	Các chỉ số đánh giá	46
3.4.1	Imbalance Score trước và sau can thiệp	46
3.4.2	Số lần migration không cần thiết (False Positive)	46
3.5	Kết quả thực nghiệm OpenstackDRS	46
3.5.1	Kết quả tổng quan	46
3.5.2	Kết quả chi tiết từng kịch bản	47
3.5.3	So sánh với phương pháp baseline	59
3.5.4	Đánh giá tổng hợp	61
3.6	Tổng kết chương	62
Kết luận		63
Tài liệu tham khảo		65

Danh sách hình vẽ

1.1	Kiến trúc các loại ảo hóa	6
1.2	Các mô hình dịch vụ và triển khai của điện toán đám mây	8
1.3	Mô hình kiến trúc các dịch vụ chính OpenStack	10
1.4	Mô hình kết hợp Prometheus và Grafana	13
1.5	Luồng xử lý dữ liệu của mô hình Amazon Chronos	16
2.1	Mô hình kiến trúc hệ thống	19
2.2	Luồng xử lý dữ liệu trong hệ thống OpenstackDRS	22
2.3	Giao diện hệ thống OpenstackDRS	31
2.4	Kết quả benchmark các mô hình dự báo chuỗi thời gian	34
3.1	Cụm Openstack được triển khai thành công trên VPS	43
3.2	Bản đồ tải khi thực hiện kịch bản 1	48
3.3	Bản đồ tải khi thực hiện kịch bản 2	49
3.4	Bản đồ tải khi thực hiện kịch bản 3	50
3.5	Bản đồ tải khi thực hiện kịch bản 4	51
3.6	Bản đồ tải khi thực hiện kịch bản 5	53
3.7	Bản đồ tải khi thực hiện kịch bản 6	55
3.8	Bản đồ tải khi thực hiện kịch bản 7	56
3.9	Bản đồ tải khi thực hiện kịch bản 8	57
3.10	Bản đồ tải khi thực hiện kịch bản 9	58

Danh sách bảng

2.1	Thống kê mô tả dữ liệu huấn luyện thu thập từ compute1	35
2.2	Siêu tham số huấn luyện TimeSeriesPredictor	36
2.3	Kết quả đánh giá trên test set (300 mẫu cuối) cho CPU Usage và Memory Usage	40
3.1	Thông tin các node trong cụm OpenStack	42
3.2	Danh sách kịch bản thực nghiệm	45
3.3	Kết quả tổng hợp các kịch bản thực nghiệm — OpenstackDRS . . .	47
3.4	S1 — Diễn biến cluster ổn định	48
3.5	S2 — Diễn biến node vượt ngưỡng đột ngột	49
3.6	S3 — Diễn biến spike ngắn hạn tự phục hồi	50
3.7	S4 — Diễn biến mất cân bằng cluster khi chưa có node vượt ngưỡng	51
3.8	S5 — Diễn biến của DRS theo pattern cao tải lặp lại	52
3.9	S6 — Diễn biến dự báo mất cân bằng trong tương lai	54
3.10	S7 — Diễn biến khi không có node đích khả thi	55
3.11	S8 — Diễn biến xử lý khi một node bị mất metrics	57
3.12	S9 — Diễn biến cooldown sau migration để tránh ping-pong	58
3.13	So sánh hành vi OpenstackDRS và OpenStack Watcher theo từng kịch bản	60
3.14	So sánh tỷ lệ FP Rate	61

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
DRS	Distributed Resource Scheduler	Bộ lập lịch tài nguyên phân tán
VM	Virtual Machine	Máy ảo
IS	Imbalance Score	Điểm mất cân bằng
CPU	Central Processing Unit	Bộ xử lý trung tâm
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
LLM	Large Language Model	Mô hình ngôn ngữ lớn
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
REST	Representational State Transfer	Kiến trúc truyền tải trạng thái biểu diễn
gRPC	Google Remote Procedure Call	Giao thức gọi hàm/thủ tục từ xa của Google
RMSE	Root Mean Square Error	Sai số căn phương trung bình
MASE	Mean Absolute Scaled Error	Sai số tuyệt đối trung bình có chuẩn hóa

Mở đầu

Đặt vấn đề

Điện toán đám mây đã trở thành nền tảng hạ tầng cốt lõi của hầu hết các tổ chức hiện đại. Các nền tảng như OpenStack được triển khai rộng rãi để vận hành hàng nghìn máy ảo (VM) trên các cụm tính toán quy mô lớn, trong đó việc phân bổ tài nguyên hiệu quả ảnh hưởng trực tiếp đến hiệu năng hệ thống, chi phí vận hành và chất lượng dịch vụ.

Tuy nhiên, Nova Scheduler – cơ chế phân bổ mặc định của OpenStack – chỉ can thiệp tại thời điểm tạo VM và không tự động điều chỉnh lại sau đó. Khi workload thay đổi theo thời gian, hệ thống dễ rơi vào trạng thái mất cân bằng tải động: một số Compute node bị quá tải trong khi các node khác lại nhàn rỗi. Để xử lý tình trạng này, người vận hành phải theo dõi thủ công và thực hiện live migration – một quy trình tốn thời gian, dễ sai sót và không thể duy trì liên tục.

Các giải pháp tự động hóa hiện có thường dựa trên ngưỡng cố định hoặc luật định sẵn, mang tính phản ứng và thiếu khả năng thích nghi. Việc tích hợp trí tuệ nhân tạo cho phép hệ thống **dự đoán trước** xu hướng sử dụng tài nguyên và **chủ động** lập kế hoạch migration, thay vì chờ quá tải xảy ra. Bên cạnh đó, bài toán còn phải thỏa mãn nhiều ràng buộc phức tạp như affinity/anti-affinity và giới hạn số migration đồng thời, khiến đây trở thành một bài toán ra quyết định đa mục tiêu thực sự.

Từ bối cảnh đó, khóa luận này xây dựng hệ thống **OpenstackDRS** (Distributed Resource Scheduler for OpenStack) – tự động phát hiện mất cân bằng tải, dự đoán trạng thái tài nguyên và lập kế hoạch live migration VM thông minh trong môi trường OpenStack.

Mô tả bài toán

Phát biểu bài toán

Bài toán đặt ra là xây dựng hệ thống OpenstackDRS có khả năng tự động phát hiện trạng thái mất cân bằng tài nguyên trong cụm OpenStack và đề xuất hoặc

thực thi kế hoạch live migration VM nhằm tái cân bằng tải, với sự kiểm soát của người vận hành.

Đầu vào

- Chuỗi dữ liệu lịch sử và thời gian thực về mức sử dụng CPU (%) RAM (%), và SWAP(%) của từng Compute node, thu thập từ Prometheus với tần suất 15 giây/lần.
- Danh sách VM đang chạy trên cụm kèm thông tin vị trí (đang chạy trên node nào) và tài nguyên đã cấp phát, truy vấn từ OpenStack Nova API.
- Tập policy do người quản trị định nghĩa: ngưỡng CPU/RAM/SWAP kích hoạt cân bằng tải, ràng buộc affinity/anti-affinity, danh sách node đang bảo trì, số lượng migration tối đa mỗi chu trình.

Đầu ra

- Dự đoán trạng thái tài nguyên của từng Compute node trong cửa sổ 75 phút tiếp theo.
- Điểm mất cân bằng (ImbalanceScore) định lượng mức độ lệch tải tại thời điểm hiện tại và dự đoán.
- Kế hoạch live migration: danh sách các cặp (VM, node_nguồn, node_đích) được sắp xếp theo thứ tự thực thi, kèm trạng thái tài nguyên dự kiến sau khi áp dụng kế hoạch.
- Kết quả thực thi: trạng thái từng migration (thành công / thất bại / bỏ qua), thời gian thực thi và ImbalanceScore thực tế sau chu trình.

Ràng buộc và giả định

- Sau khi thực thi kế hoạch, không có Compute node nào vượt ngưỡng CPU, RAM hoặc SWAP an toàn đã định.

- Kế hoạch phải tôn trọng toàn bộ policy: VM bị ràng buộc anti-affinity với VM khác không được di chuyển đến cùng node; các node đang bảo trì không được chọn làm đích.
- Số lượng live migration trong một chu trình không vượt quá giới hạn cấu hình, nhằm kiểm soát băng thông mạng và tránh ảnh hưởng đồng thời đến nhiều VM.
- Hệ thống không được để cụm rơi vào trạng thái tồi hơn trước khi can thiệp – nếu kế hoạch mô phỏng không cải thiện ImbalanceScore, chu trình bị hủy và không có migration nào được thực thi.
- Hệ thống được xây dựng và kiểm thử trên cụm OpenStack phiên bản Caracal gồm một Controller node và ba Compute node với tổng cộng 10 VM. Chỉ số tài nguyên được sử dụng là CPU, RAM và SWAP; các chỉ số I/O disk và băng thông mạng nằm ngoài phạm vi khóa luận.

Mục tiêu và phạm vi nghiên cứu

Mục tiêu nghiên cứu

Đề tài hướng đến các mục tiêu cụ thể sau:

- Xây dựng module dự đoán tài nguyên có khả năng dự báo mức sử dụng CPU và RAM của từng Compute node trong cửa sổ thời gian 10 phút tiếp theo, dựa trên chuỗi dữ liệu lịch sử.
- Xây dựng cơ chế lập kế hoạch migration thông minh: tự động xác định các VM cần di chuyển, chọn node đích phù hợp và mô phỏng trạng thái cụm sau khi áp dụng kế hoạch, đảm bảo các ràng buộc affinity/anti-affinity.
- Tích hợp toàn bộ luồng xử lý – từ thu thập dữ liệu, dự đoán, lập kế hoạch đến thực thi – thành một hệ thống hoàn chỉnh vận hành trên OpenStack thực tế.
- Đánh giá hiệu quả của hệ thống thông qua các chỉ số định lượng: ImbalanceScore trước và sau khi can thiệp, tỷ lệ migration thành công và mức độ cải thiện phân bố tài nguyên.

Phạm vi nghiên cứu

Đề tài tập trung vào các nội dung sau:

- Nghiên cứu và áp dụng các mô hình dự đoán chuỗi thời gian phù hợp với dữ liệu tài nguyên VM.
- Thiết kế và cài đặt thuật toán lập kế hoạch migration có xét đến các ràng buộc policy phức tạp.
- Xây dựng hệ thống OpenstackDRS với kiến trúc microservice, tích hợp với OpenStack Nova API và hệ thống giám sát Prometheus.
- Cung cấp Dashboard cho phép người vận hành theo dõi trạng thái cụm, xem xét kế hoạch migration và lựa chọn chế độ vận hành (tự động hoặc phê duyệt thủ công).

Các vấn đề sau nằm ngoài phạm vi của đề tài: cân bằng tải theo chỉ số I/O disk và băng thông mạng; tối ưu hóa tiêu thụ điện năng bằng cách hợp nhất VM và tắt node. Những hướng mở rộng này được đề xuất trong phần kết luận.

Cấu trúc khóa luận

Cấu trúc của khóa luận được tổ chức như sau:

Chương 1 – Cơ sở lý thuyết trình bày các kiến thức nền tảng liên quan đến môi trường OpenStack, cơ chế live migration, các mô hình dự đoán chuỗi thời gian và phương pháp lập kế hoạch tối ưu được sử dụng trong đề tài.

Chương 2 – Phân tích và thiết kế hệ thống mô tả kiến trúc tổng thể của OpenstackDRS, bao gồm các thành phần chính, luồng dữ liệu và cơ chế tích hợp với hạ tầng OpenStack.

Chương 3 – Thực nghiệm và đánh giá trình bày kết quả thực nghiệm, phân tích hiệu quả của hệ thống qua các kịch bản kiểm thử và so sánh với phương pháp baseline.

Chương 1

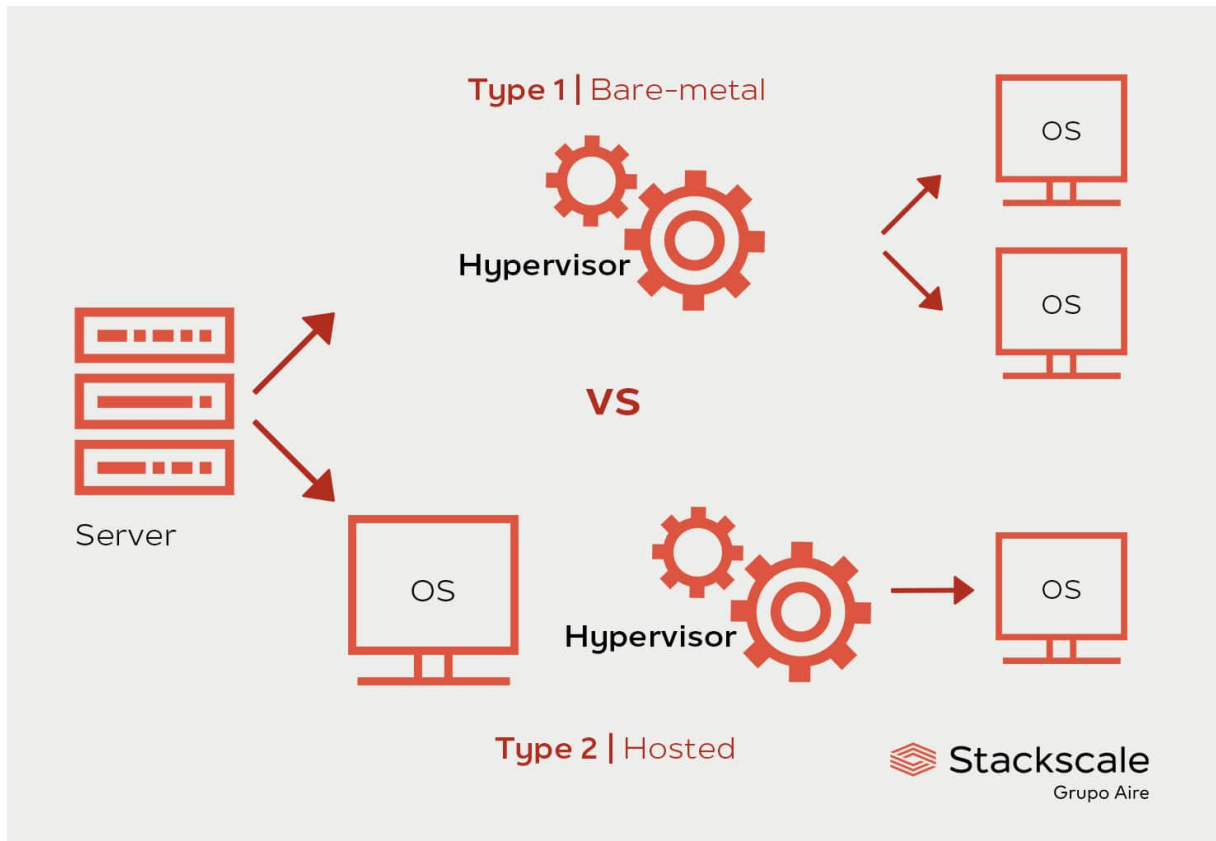
Cơ sở lý thuyết

1.1 Ảo hóa và điện toán đám mây

1.1.1 Ảo hóa

Ảo hóa (Virtualization) là một công nghệ sử dụng phần mềm để tạo ra một lớp trừu tượng trên phần cứng vật lý, cho phép một máy vật lý (máy chủ, máy tính) được chia thành nhiều máy ảo (VM) độc lập. Mỗi máy ảo chạy hệ điều hành và ứng dụng riêng của nó, đồng thời chia sẻ CPU, bộ nhớ và lưu trữ bên dưới, giúp nâng cao hiệu quả sử dụng tài nguyên, giảm chi phí phần cứng và tăng khả năng mở rộng. [1]

Thành phần trung tâm của ảo hóa là **hypervisor** (còn gọi là Virtual Machine Monitor – VMM), một lớp phần mềm chịu trách nhiệm tạo, quản lý và phân bổ tài nguyên phần cứng cho các máy ảo. Có hai loại hypervisor phổ biến: loại 1 (bare-metal) chạy trực tiếp trên phần cứng vật lý như VMware ESXi, Microsoft Hyper-V và KVM; loại 2 (hosted) chạy trên hệ điều hành máy chủ như VMware Workstation và VirtualBox [2]. Chi tiết như trên hình 1.1



Hình 1.1: Kiến trúc các loại ảo hóa. Nguồn: Stackscale [2]

Ảo hóa mang lại nhiều lợi ích thiết thực trong triển khai hạ tầng hiện đại. Đầu tiên là cô lập: mỗi máy ảo hoạt động độc lập, sự cố trên một VM không ảnh hưởng đến các VM khác. Thứ hai là tính linh động: VM có thể được di chuyển, sao chép hoặc khôi phục một cách dễ dàng. Thứ ba là tối ưu hóa tài nguyên: một máy chủ vật lý có thể chạy nhiều VM, giảm thiểu tình trạng lãng phí tài nguyên phần cứng nhàn rỗi.

1.1.2 Điện toán đám mây

Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) định nghĩa điện toán đám mây (Cloud Computing) là “mô hình cho phép truy cập mạng theo yêu cầu, thuận tiện và phổ biến đến một nhóm tài nguyên điện toán có thể cấu hình được (ví dụ: mạng, máy chủ, lưu trữ, ứng dụng và dịch vụ) có thể được cung cấp và thu hồi nhanh chóng với nỗ lực quản lý tối thiểu hoặc tương tác tối thiểu với nhà cung cấp dịch vụ” [3].

Theo NIST, mô hình đám mây bao gồm năm đặc điểm thiết yếu, ba mô hình dịch

vụ và bốn mô hình triển khai [3]:

Năm đặc điểm thiết yếu

- **Tự phục vụ theo yêu cầu (On-demand self-service):** Người dùng có thể tự cấp phát tài nguyên điện toán mà không cần tương tác với nhà cung cấp.
- **Truy cập mạng rộng (Broad network access):** Tài nguyên có thể truy cập qua mạng thông qua các cơ chế chuẩn.
- **Gộp tài nguyên (Resource pooling):** Tài nguyên của nhà cung cấp được tổng hợp để phục vụ nhiều người dùng theo mô hình đa thuê bao.
- **Co giãn nhanh (Rapid elasticity):** Tài nguyên có thể được cấp phát và thu hồi linh hoạt, tự động theo nhu cầu.
- **Dịch vụ đo lường được (Measured service):** Hệ thống đám mây tự động kiểm soát và tối ưu hóa việc sử dụng tài nguyên thông qua các cơ chế đo lường.

Ba mô hình dịch vụ

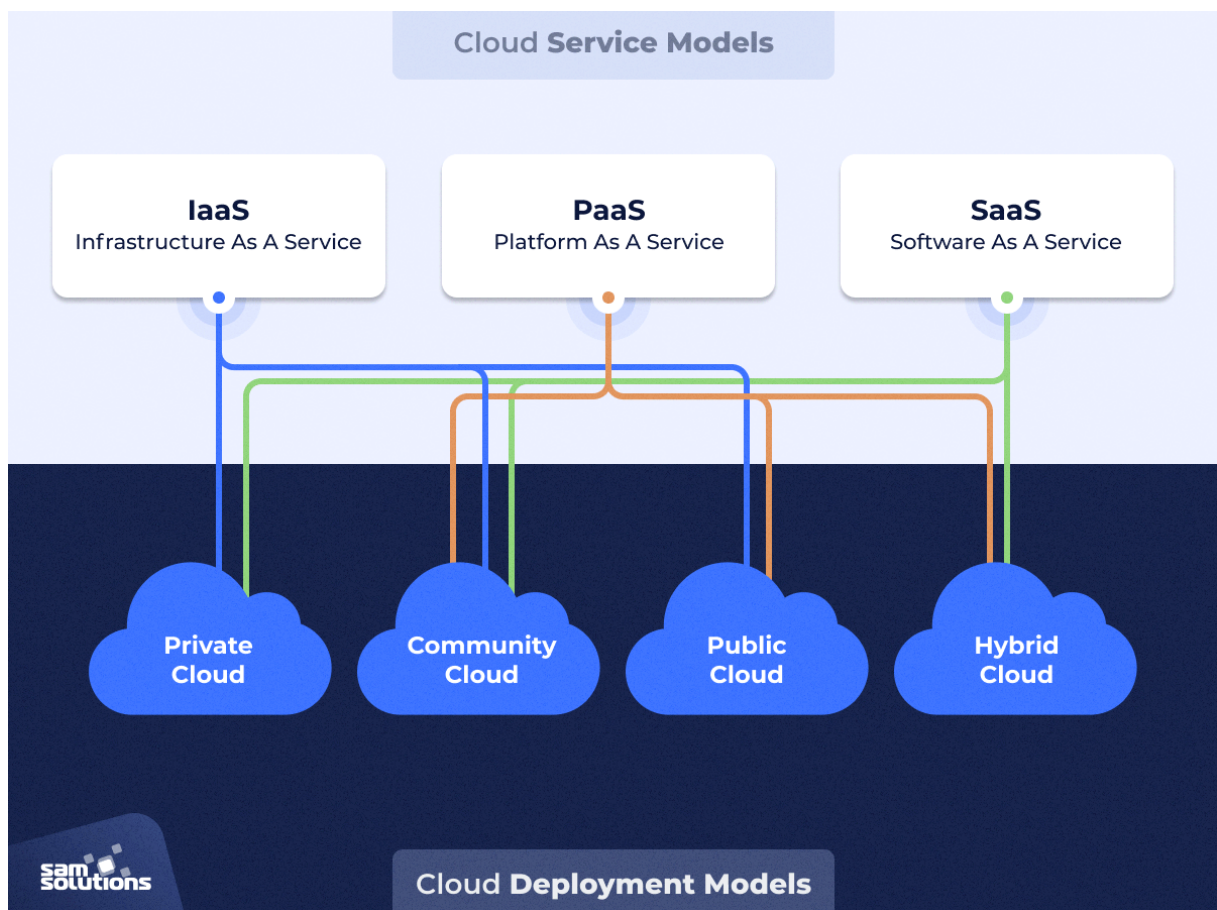
NIST phân loại ba mô hình dịch vụ đám mây chính như hình 1.2:

- **IaaS (Infrastructure as a Service):** Cung cấp quyền truy cập vào các tài nguyên hạ tầng điện toán cơ bản như tính toán, mạng và lưu trữ. Người dùng kiểm soát hệ điều hành, lưu trữ và ứng dụng đã triển khai.
- **PaaS (Platform as a Service):** Cung cấp nền tảng phát triển và triển khai ứng dụng, bao gồm cơ sở dữ liệu, middleware và môi trường thực thi. Người dùng không cần quản lý hạ tầng bên dưới.
- **SaaS (Software as a Service):** Người dùng sử dụng ứng dụng của nhà cung cấp chạy trên hạ tầng đám mây thông qua giao diện web hoặc API, không cần quản lý bất kỳ thành phần hạ tầng nào.

Bốn mô hình triển khai

NIST phân loại bốn mô hình triển khai đám mây chính như hình 1.2:

- **Đám mây riêng (Private cloud):** Hạ tầng đám mây được triển khai và sử dụng cho một tổ chức duy nhất.
- **Đám mây cộng đồng (Community cloud):** Hạ tầng được chia sẻ giữa một số tổ chức có cùng mối quan tâm.
- **Đám mây công cộng (Public cloud):** Hạ tầng được cung cấp cho công chúng rộng rãi bởi nhà cung cấp dịch vụ.
- **Đám mây lai (Hybrid cloud):** Kết hợp hai hoặc nhiều mô hình triển khai đám mây khác nhau [3].



Hình 1.2: Các mô hình dịch vụ và triển khai của điện toán đám mây. Nguồn: [4]

1.1.3 Đám mây OpenStack

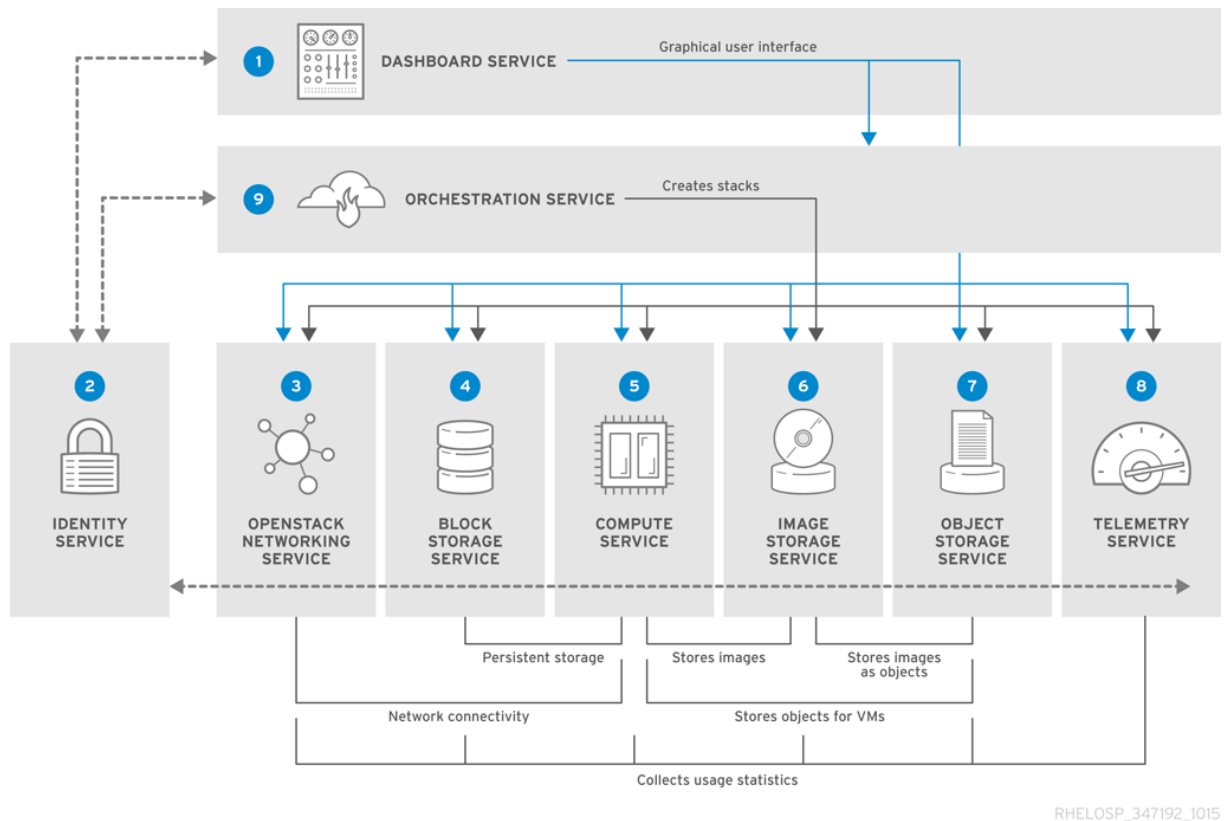
OpenStack là một nền tảng điện toán đám mây mã nguồn mở cho phép xây dựng và quản lý môi trường đám mây riêng (private cloud) và đám mây công cộng

(public cloud). Ban đầu được phát triển bởi NASA và Rackspace vào năm 2010, OpenStack đã trở thành một trong những dự án cơ sở hạ tầng đám mây mã nguồn mở lớn nhất thế giới với sự đóng góp từ hàng trăm tổ chức và hàng nghìn lập trình viên [5].

OpenStack sử dụng kiến trúc mô-đun (modular architecture), trong đó mỗi dịch vụ chịu trách nhiệm cho một tính năng đám mây riêng biệt, bao gồm quản lý danh tính, danh mục ảnh hệ thống, cũng như các chức năng tính toán, mạng và lưu trữ [6]. Các thành phần cốt lõi của OpenStack bao gồm:

- **Nova (Compute):** Quản lý và cung cấp số lượng lớn máy ảo, đảm bảo tài nguyên tính toán cho các ứng dụng.
- **Neutron (Networking):** Cung cấp mô hình “Networking-as-a-Service”, cho phép người dùng xây dựng các cấu trúc liên kết mạng phức tạp.
- **Cinder (Block Storage):** Cung cấp các ổ đĩa lưu trữ khối (block storage) bền vững có thể gắn vào các instance đang chạy.
- **Glance (Image Service):** Quản lý danh mục ảnh đĩa (disk image) dùng để khởi tạo máy ảo.
- **Keystone (Identity):** Cung cấp dịch vụ xác thực và phân quyền tập trung cho toàn bộ hệ thống OpenStack.
- **Swift (Object Storage):** Cung cấp lưu trữ đối tượng có khả năng mở rộng cao cho dữ liệu phi cấu trúc.
- **Horizon (Dashboard):** Giao diện web cho phép người dùng quản lý, cung cấp và giám sát tài nguyên đám mây.

OpenStack được thiết kế theo kiến trúc phân tán (distributed architecture) như trên hình 1.3, cho phép mở rộng hạ tầng đám mây theo chiều ngang trên nhiều máy chủ vật lý, đồng thời tuân theo mô hình hướng API (API-driven), giúp tự động hóa và tích hợp với các công cụ bên ngoài trở nên dễ dàng [7].



Hình 1.3: Mô hình kiến trúc các dịch vụ chính OpenStack. Nguồn: Red Hat [6]

1.2 Giám sát hệ thống

1.2.1 Prometheus

Prometheus là một hệ thống giám sát và cảnh báo mã nguồn mở được phát triển tại SoundCloud từ năm 2012, sau khi các công cụ giám sát hiện có không đáp ứng được yêu cầu của hạ tầng container hóa. Năm 2016, Prometheus được Cloud Native Computing Foundation (CNCF) tiếp nhận và trở thành dự án trưởng thành (graduation project) thứ hai của CNCF vào năm 2018, sau Kubernetes [8].

Kiến trúc của Prometheus được xây dựng xung quanh mô hình kéo (pull model): máy chủ Prometheus định kỳ thu thập (scrape) dữ liệu từ các mục tiêu (targets) đã được cấu hình, thay vì chờ dữ liệu được đẩy về [9]. Mô hình này mang lại tính linh hoạt và tích hợp tốt với môi trường hạ tầng động. Các thành phần chính của hệ thống Prometheus bao gồm:

- **Prometheus Server:** Thành phần trung tâm chịu trách nhiệm thu thập, lưu

trữ dữ liệu chuỗi thời gian và đánh giá các quy tắc cảnh báo.

- **Time Series Database (TSDB):** Cơ sở dữ liệu chuỗi thời gian tích hợp, nơi lưu trữ toàn bộ dữ liệu metrics. Dữ liệu gần đây được lưu kết hợp trong bộ nhớ và các tệp được ánh xạ bộ nhớ (mmap), trong khi dữ liệu cũ hơn được ghi vào các khối lưu trữ lâu dài.
- **Exporter:** Các thành phần chạy trên các máy chủ được giám sát, thu thập và phơi bày (expose) các metrics theo định dạng Prometheus.
- **Alertmanager:** Thành phần xử lý cảnh báo được sinh ra từ máy chủ Prometheus và gửi thông báo qua email, Slack hoặc các kênh khác.
- **Service Discovery:** Cơ chế tự động phát hiện các mục tiêu cần giám sát trong môi trường hạ tầng động như Kubernetes [9].

Prometheus sử dụng **mô hình dữ liệu đa chiều (multi-dimensional data model)**, trong đó mỗi chuỗi thời gian được xác định bởi tên metric và một tập hợp các cặp khóa-giá trị (labels). Ngôn ngữ truy vấn PromQL (Prometheus Query Language) cung cấp khả năng lựa chọn, tổng hợp và chuyển đổi dữ liệu chuỗi thời gian một cách mạnh mẽ, hỗ trợ các hàm như `rate()`, instant vector và range vector [8]. Prometheus định nghĩa bốn loại metric cơ bản: Counter (giá trị đơn điệu tăng), Gauge (giá trị có thể tăng hoặc giảm), Histogram (phân phối mẫu quan sát) và Summary.

1.2.2 Grafana

Grafana là nền tảng trực quan hóa và phân tích dữ liệu mã nguồn mở hàng đầu, cho phép người dùng truy vấn, trực quan hóa, thiết lập cảnh báo và hiểu dữ liệu từ nhiều nguồn khác nhau [10]. Được phát hành lần đầu vào tháng 1 năm 2014, Grafana ban đầu nhắm tới Graphite và InfluxDB làm nguồn dữ liệu, sau đó mở rộng hỗ trợ đa dạng nguồn dữ liệu bao gồm Prometheus, AWS CloudWatch, Elasticsearch, PostgreSQL và MySQL [11].

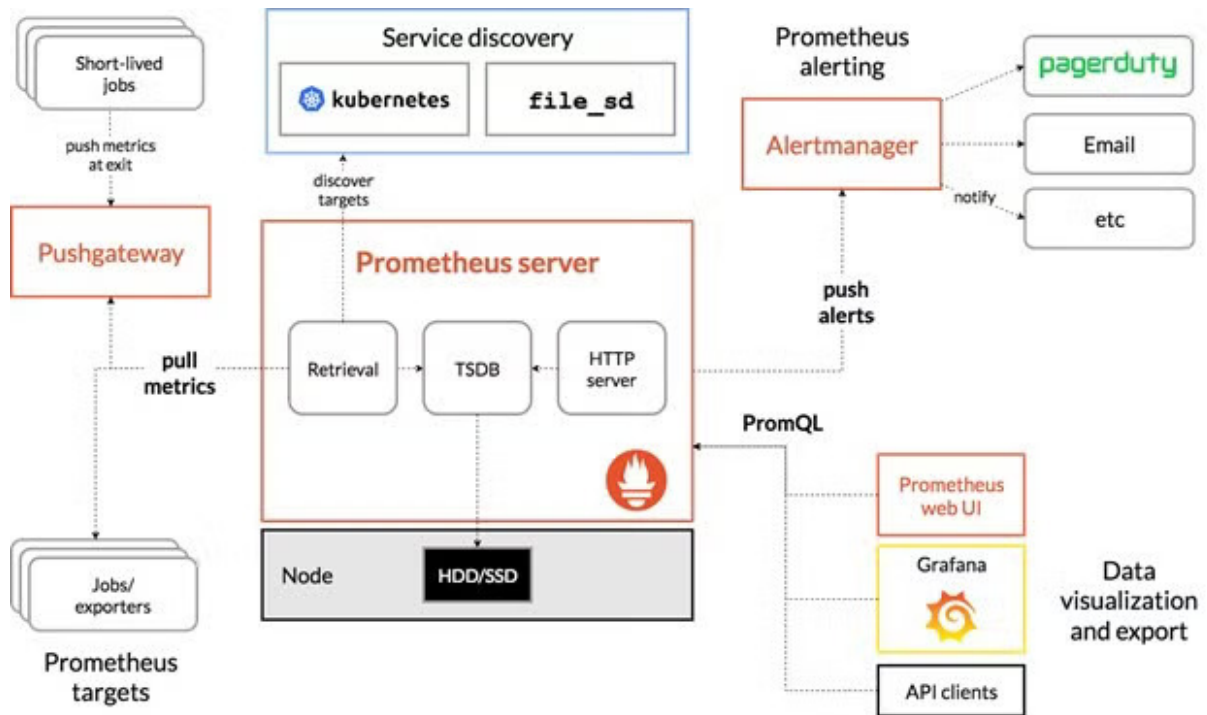
Điểm đặc biệt của Grafana là nó không tự lưu trữ dữ liệu metrics, mà kết nối đến các nguồn dữ liệu ngoài (external data sources) và truy vấn chúng tại thời điểm hiển thị, đồng thời có thể kết hợp dữ liệu từ nhiều nguồn trên cùng một

dashboard. Kiến trúc phần mềm của Grafana được viết bằng Go (backend) và TypeScript/React (frontend), chạy dưới dạng một tệp nhị phân duy nhất phục vụ giao diện web [11].

Các tính năng nổi bật của Grafana bao gồm:

- **Dashboard linh hoạt:** Người dùng có thể tạo các dashboard động và có thể tái sử dụng với các biến mẫu (template variables), hỗ trợ nhiều loại trực quan hóa như biểu đồ thời gian, heatmap, histogram và bản đồ địa lý.
- **Hỗ trợ đa nguồn dữ liệu:** Grafana hỗ trợ hơn 50 nguồn dữ liệu khác nhau và cho phép kết hợp nhiều nguồn trên cùng một biểu đồ.
- **Hệ thống cảnh báo:** Người dùng có thể định nghĩa các quy tắc cảnh báo dựa trên ngưỡng và gửi thông báo đến Slack, PagerDuty, email và nhiều kênh khác.
- **Hệ sinh thái plugin:** Grafana cung cấp một hệ sinh thái plugin phong phú, bao gồm plugin nguồn dữ liệu, plugin bảng điều khiển và plugin ứng dụng, mở rộng khả năng tích hợp với các dịch vụ bên ngoài.
- **Chú thích đồ thị (Annotations):** Cho phép đánh dấu các sự kiện quan trọng trực tiếp trên đồ thị từ nhiều nguồn dữ liệu, hỗ trợ phân tích tương quan giữa sự kiện và biến động metrics [12].

Trong thực tế, Prometheus và Grafana thường được triển khai kết hợp với nhau: Prometheus đóng vai trò thu thập và lưu trữ metrics, trong khi Grafana đảm nhiệm việc trực quan hóa và phân tích dữ liệu đó thông qua các dashboard trực quan như trên hình 1.4. Sự kết hợp này đã trở thành bộ công cụ giám sát tiêu chuẩn cho các hệ thống cloud-native và Kubernetes.



Hình 1.4: Mô hình kết hợp Prometheus và Grafana. Nguồn: CNCF [13]

1.3 Bài toán dự đoán chuỗi thời gian và Amazon-Chronos

1.3.1 Bài toán dự đoán trong hệ thống

Trong bối cảnh các hệ thống thông minh hiện đại, bài toán dự đoán chuỗi thời gian đóng vai trò cốt lõi trong việc hỗ trợ ra quyết định, tối ưu vận hành và nâng cao trải nghiệm người dùng. Chuỗi thời gian (time series) là tập hợp các quan sát được ghi nhận theo thứ tự thời gian, ví dụ như lưu lượng truy cập hệ thống, giá cổ phiếu, nhu cầu tiêu thụ điện năng, hay số lượng người dùng hoạt động theo từng giờ/ngày.

Trong hệ thống đề xuất, bài toán dự đoán được đặt ra nhằm ước lượng giá trị trong tương lai dựa trên dữ liệu lịch sử đã quan sát. Cụ thể, giả sử ta có chuỗi dữ liệu $X = \{x_1, x_2, \dots, x_t\}$, mục tiêu là xây dựng một hàm dự đoán f sao cho:

$$\hat{x}_{t+h} = f(x_1, x_2, \dots, x_t)$$

trong đó h là khoảng thời gian dự báo (forecast horizon), và $\hat{x}_{t+h}$ là giá trị dự đoán tại thời điểm tương lai.

Bài toán này gặp nhiều thách thức như:

- **Tính không dừng (non-stationarity):** Dữ liệu có thể thay đổi xu hướng hoặc mùa vụ theo thời gian.
- **Nhiều và biến động mạnh:** Các yếu tố ngẫu nhiên hoặc sự kiện bất thường có thể ảnh hưởng lớn đến dữ liệu.
- **Phụ thuộc dài hạn:** Một số chuỗi có sự phụ thuộc giữa các điểm dữ liệu cách xa nhau.

Do đó, việc lựa chọn mô hình phù hợp để học được các đặc trưng này là yếu tố quyết định đến độ chính xác của hệ thống dự đoán.

1.3.2 Mô hình ngôn ngữ lớn và Amazon Chronos

Mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn (Large Language Model – LLM) là một lớp mô hình học sâu được xây dựng trên kiến trúc Transformer, được huấn luyện trên lượng dữ liệu văn bản khổng lồ nhằm mô hình hóa phân phối xác suất của ngôn ngữ tự nhiên [14]. Về bản chất, LLM học cách dự đoán token tiếp theo trong một chuỗi dựa trên ngữ cảnh đã cho, tức là ước lượng xác suất có điều kiện:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, x_2, \dots, x_{t-1}) \quad (1.1)$$

Thành phần cốt lõi tạo nên sức mạnh của LLM là cơ chế *self-attention* đa đầu (Multi-Head Self-Attention). Cho một chuỗi đầu vào được biểu diễn dưới dạng ma trận $X \in \mathbb{R}^{n \times d}$, cơ chế attention được tính theo công thức:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (1.2)$$

trong đó $Q = XW^Q$, $K = XW^K$, $V = XW^V$ là các ma trận truy vấn (query), khóa (key) và giá trị (value) được chiếu tuyến tính từ đầu vào; d_k là chiều của không gian khóa dùng để chuẩn hóa tránh bão hòa gradient. Cơ chế này cho phép mỗi token trong chuỗi attend đến tất cả các token còn lại, nắm bắt các phụ thuộc tầm xa mà không cần tính toán hồi quy tốn kém – một ưu điểm vượt trội so với các kiến trúc RNN và LSTM trước đó [15].

Quá trình huấn luyện LLM thường diễn ra qua hai giai đoạn: *pre-training* trên tập dữ liệu lớn và không có nhãn thông qua mục tiêu tự giám sát, sau đó là *fine-tuning* để thích nghi với các tác vụ cụ thể [14]. Nhờ vào khả năng học biểu diễn phổ quát từ dữ liệu quy mô lớn, LLM có thể thực hiện nhiều tác vụ khác nhau – bao gồm cả các tác vụ ngoài phạm vi ngôn ngữ tự nhiên – mà không cần huấn luyện lại từ đầu.

Amazon Chronos

Amazon Chronos [16] là một họ mô hình nền tảng (foundation model) dành cho bài toán dự báo chuỗi thời gian, được xây dựng dựa trên ý tưởng cốt lõi là xem chuỗi thời gian như một ngôn ngữ và tận dụng trực tiếp kiến trúc Transformer của LLM để mô hình hóa. Thay vì thiết kế kiến trúc chuyên biệt, Chronos tái sử dụng các mô hình ngôn ngữ hiện có (T5, GPT-2) và thực hiện hai bước xử lý để biến dữ liệu số thành chuỗi token rời rạc.

Bước 1 – Chuẩn hóa và lượng tử hóa. Chuỗi thời gian $\{z_1, z_2, \dots, z_T\}$ được chuẩn hóa theo giá trị trung bình tuyệt đối:

$$\tilde{z}_t = \frac{z_t}{\mu}, \quad \mu = \frac{1}{T} \sum_{t=1}^T |z_t| \quad (1.3)$$

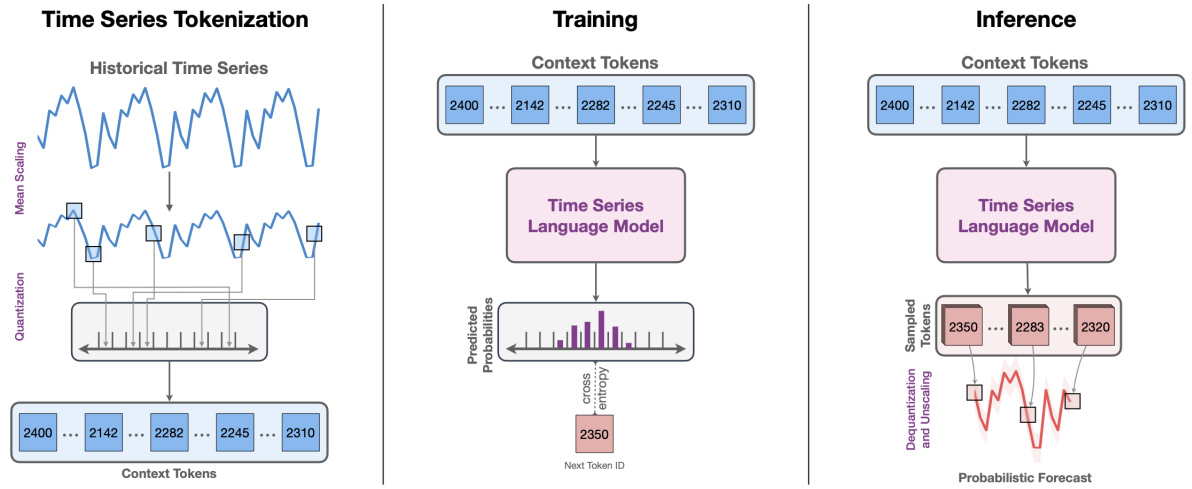
Sau đó, các giá trị đã chuẩn hóa được lượng tử hóa thành B bins đều nhau trong khoảng $[-C, C]$, ánh xạ mỗi giá trị thực sang một token rời rạc $c_t \in \{1, 2, \dots, B\}$. Cách tiếp cận này biến bài toán hồi quy thành bài toán phân loại, cho phép mô hình học phân phối xác suất tùy ý thay vì chỉ dự báo điểm [16].

Bước 2 – Mô hình hóa và sinh dự báo. Chuỗi token đã lượng tử hóa được đưa vào LLM, huấn luyện bằng hàm mất mát cross-entropy phân loại:

$$\mathcal{L} = - \sum_t \log P(c_t \mid c_1, c_2, \dots, c_{t-1}) \quad (1.4)$$

Tại thời điểm suy luận, mô hình lấy mẫu nhiều chuỗi token tương lai, giải mã ngược về giá trị thực và cung cấp dự báo xác suất dưới dạng phân vị (quantile forecasts). Chronos được huấn luyện trước trên tập dữ liệu chuỗi thời gian đa miền quy mô lớn, kết hợp tăng cường dữ liệu tổng hợp bằng quá trình Gaussian, qua đó đạt khả

năng *zero-shot forecasting* – dự báo chính xác trên các tập dữ liệu chưa từng thấy trong huấn luyện mà không cần tinh chỉnh thêm [16].



Hình 1.5: Luồng xử lý dữ liệu của mô hình Amazon Chronos. Nguồn: HuggingFace [17]

1.4 Tổng kết chương

Chương này đã trình bày các cơ sở lý thuyết quan trọng làm nền tảng cho đề tài, bao gồm các lý thuyết về ảo hóa, điện toán đám mây và nền tảng OpenStack. Bên cạnh đó, chương cũng giới thiệu các công cụ giám sát hệ thống như Prometheus và Grafana, đóng vai trò thu thập, lưu trữ và trực quan hóa dữ liệu tài nguyên. Các khái niệm về bài toán dự đoán chuỗi thời gian được trình bày nhằm làm rõ nhu cầu dự báo tải tài nguyên trong hệ thống. Chương cũng phân tích mô hình Amazon Chronos, một mô hình nền tảng dựa trên kiến trúc Transformer, có khả năng xử lý dữ liệu chuỗi thời gian thông qua cơ chế lượng tử hóa và dự báo xác suất. Những nội dung này tạo cơ sở lý luận cho việc thiết kế hệ thống OpenstackDRS có khả năng giám sát, phân tích và dự đoán trạng thái tài nguyên trong các chương tiếp theo.

Chương 2

Phân tích và thiết kế hệ thống

2.1 Yêu cầu hệ thống

2.1.1 Yêu cầu chức năng

Thu thập và tiền xử lý metrics. Hệ thống thu thập hai nhóm chỉ số chính từ Prometheus: mức sử dụng CPU (%), RAM (%) và SWAP (%) của từng compute node và của từng VM đang chạy trên cụm. Dữ liệu được truy vấn theo chu kỳ định kỳ thông qua Prometheus HTTP API, đồng thời hỗ trợ truy xuất dữ liệu lịch sử (range query) phục vụ huấn luyện mô hình. Sau khi thu thập, hệ thống chuẩn hóa định dạng, loại bỏ các điểm dữ liệu thiếu hoặc bất hợp lệ, và lưu trữ vào cơ sở dữ liệu chuỗi thời gian để phục vụ truy vấn nhanh ở các bước sau.

Dự đoán trạng thái tài nguyên. Dựa trên chuỗi dữ liệu lịch sử của từng node, module Predictor áp dụng mô hình thống kê hoặc học máy để dự báo mức sử dụng tài nguyên trong cửa sổ thời gian kế tiếp. Đầu ra của bước này là trạng thái của các compute trong khoảng thời gian tiếp theo. Kết quả dự đoán được chuyển sang module Scoring để lượng hóa mức độ mất cân bằng.

Tính điểm mất cân bằng (Scoring). Module Scoring nhận đầu vào là trạng thái hiện tại và dự đoán của toàn cụm, sau đó tính toán các chỉ số định lượng phản ánh mức độ mất cân bằng tài nguyên, chẳng hạn độ lệch chuẩn về mức sử dụng CPU/RAM giữa các node, tỉ lệ node đang hoặc sắp vượt ngưỡng. Các chỉ số này được sử dụng làm đầu vào cho Engine Service để quyết định có cần kích hoạt chu trình migration hay không.

Ra quyết định và lập kế hoạch migration. Engine Service tổng hợp kết quả Scoring, trạng thái hiện tại của cụm và các policy được người quản trị định nghĩa sẵn để xác định tập VM cần di chuyển và đích đến phù hợp. Quá trình lập kế hoạch phải đảm bảo: (i) node đích có đủ CPU và RAM để tiếp nhận VM sau migration; (ii) không vi phạm ràng buộc affinity/anti-affinity; (iii) một vm không được chọn làm ứng viên trong cùng 1 chu kỳ hoặc trong 2 chu kỳ liên tiếp. Hệ thống mô

phồng trạng thái cụm sau khi áp dụng kế hoạch trước khi đưa vào thực thi.

Thực thi live migration. Executor gọi OpenStack Nova API để thực hiện live migration theo kế hoạch đã được duyệt. Mỗi thao tác migration được thực hiện tuần tự hoặc song song có kiểm soát, kèm theo kiểm tra trạng thái trước và sau migration. Nếu một bước thất bại, hệ thống ghi nhận ký lỗi, dừng hoặc bỏ qua các bước phụ thuộc và thông báo cho người quản trị. Toàn bộ lịch sử thực thi được lưu lại để phục vụ kiểm toán và phân tích sau này.

Giao diện quản lý và Dashboard. Hệ thống cung cấp API REST cho phép người quản trị cấu hình policy, kích hoạt hoặc tắt chu trình tự động, và truy vấn lịch sử migration. Dashboard trực quan hóa trạng thái tài nguyên của từng node theo thời gian thực, kết quả dự đoán, và lịch sử các chu trình cân bằng tải đã thực hiện.

2.1.2 Yêu cầu phi chức năng

Hiệu năng và độ trễ. Chu trình thu thập – phân tích – ra quyết định cần hoàn thành trong thời gian đủ ngắn để kế hoạch migration phản ánh đúng trạng thái cụm tại thời điểm thực thi. Hệ thống phải xử lý được số liệu của cụm có hàng trăm VM mà không gây nghẽn cổ chai tại bước thu thập hoặc tính điểm.

Độ tin cậy. Việc thực thi live migration tác động trực tiếp đến VM đang chạy, do đó Executor phải có cơ chế kiểm tra điều kiện tiên quyết trước mỗi thao tác và xử lý lỗi rõ ràng. Hệ thống không được để cụm rơi vào trạng thái tồi hơn ban đầu sau một chu trình cân bằng tải.

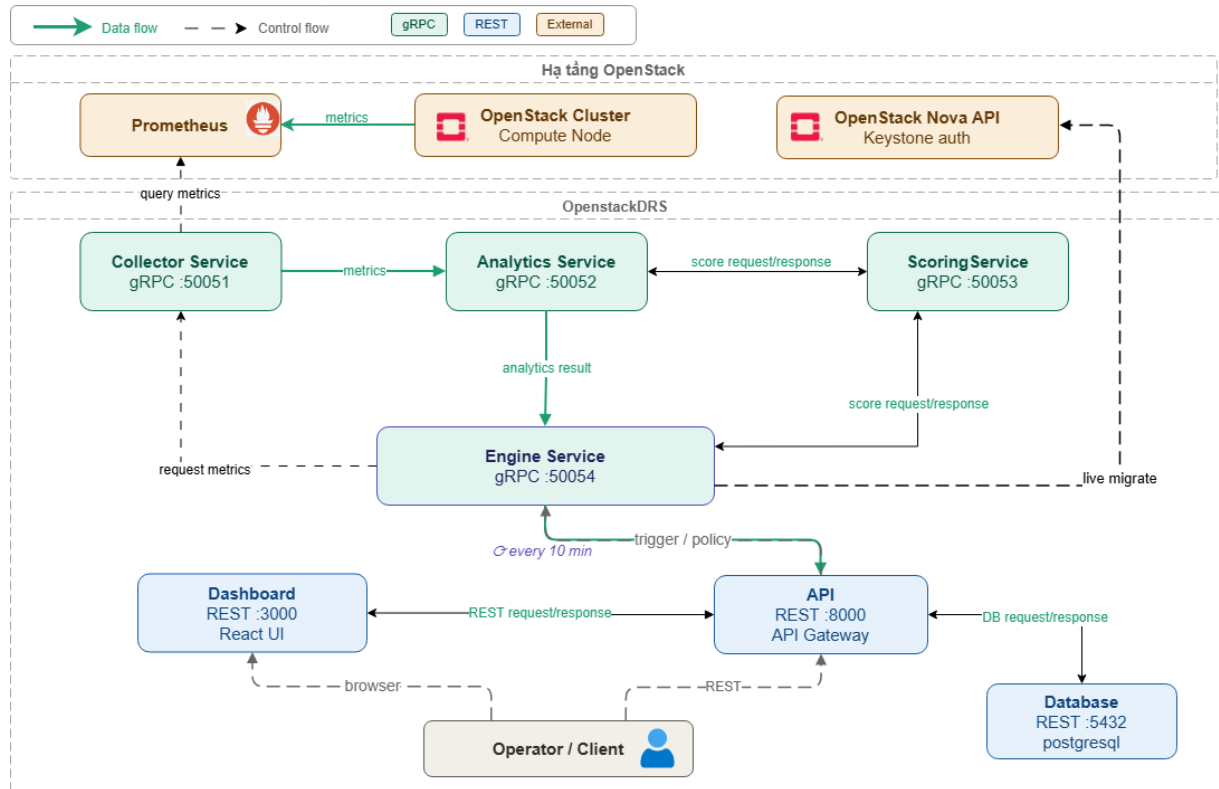
Khả năng mở rộng. Kiến trúc hướng dịch vụ cho phép mở rộng độc lập từng thành phần (Collector, Predictor, Scoring, Engine Service, Executor) khi quy mô cụm tăng lên mà không cần thay đổi toàn bộ hệ thống.

Bảo mật. Các API giao tiếp với OpenStack Nova và Prometheus cần được xác thực và phân quyền phù hợp. Thông tin xác thực (credentials) không được lưu dạng plaintext trong mã nguồn hoặc cấu hình không mã hóa.

Khả năng quan sát. Toàn bộ các bước trong vòng lặp kín – từ thu thập dữ liệu, kết quả dự đoán, điểm mất cân bằng, kế hoạch migration, đến trạng thái thực thi – đều phải được ghi log đầy đủ và có thể tra cứu lại, phục vụ việc debug và đánh giá hiệu quả của hệ thống sau triển khai.

2.2 Kiến trúc tổng quan hệ thống

2.2.1 Mô hình kiến trúc



Hình 2.1: Mô hình kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình microservices kết hợp với pipeline xử lý dữ liệu, trong đó các thành phần được tổ chức thành các tầng xử lý tuần tự nhưng vẫn đảm bảo tính độc lập và linh hoạt. Danh sách các thành phần:

- Collector Service
- Analytics Service
- Scoring Service
- Engine Service
- API
- Dashboard

- Database

Kiến trúc microservices chia hệ thống thành các dịch vụ nhỏ, mỗi dịch vụ phụ trách một chức năng riêng trong quy trình xử lý dữ liệu như thu thập, phân tích, ra quyết định và thực thi. Cách tiếp cận này giúp giảm phụ thuộc giữa các thành phần, dễ nâng cấp, mở rộng và thay thế từng phần mà không ảnh hưởng toàn hệ thống.

Hệ thống cũng áp dụng mô hình pipeline, trong đó dữ liệu được xử lý tuần tự qua nhiều giai đoạn. Điều này giúp chuẩn hóa luồng xử lý, dễ theo dõi, debug và cho phép scale độc lập từng bước. Việc tách rõ các tầng xử lý còn giúp hệ thống dễ tích hợp thêm công nghệ mới.

2.2.2 Luồng xử lý dữ liệu

Luồng dữ liệu trong hệ thống được tổ chức theo mô hình pipeline tuần tự, trong đó mỗi giai đoạn đảm nhận một vai trò chuyên biệt và độc lập. Thiết kế này đảm bảo tính minh bạch trong xử lý, đồng thời cho phép thay thế hoặc nâng cấp từng giai đoạn mà không ảnh hưởng đến phần còn lại của hệ thống.

Giai đoạn 1 — Thu thập dữ liệu: Pipeline bắt đầu từ Prometheus, nơi các metrics phản ánh trạng thái tài nguyên (CPU, RAM, SWAP) được thu thập liên tục từ các compute node. Collector Service truy vấn Prometheus theo chu kỳ cố định, đảm bảo dữ liệu đầu vào luôn cập nhật và phản ánh đúng tình trạng thực tế của hệ thống tại từng thời điểm.

Giai đoạn 2 — Tiền xử lý và chuẩn hóa dữ liệu. Dữ liệu thô từ Prometheus được làm sạch, chuẩn hóa về định dạng thống nhất và kiểm tra tính hợp lệ để làm nguồn dữ liệu chuẩn phù hợp phục vụ cho xử lý của Analytics Service và đầu vào của mô hình Amazon Chronos.

Giai đoạn 3 — Phân tích và dự báo: Dữ liệu chuỗi thời gian đã chuẩn hóa được đưa vào Analytics Service để thực hiện hai nhiệm vụ song song: phát hiện bất thường (anomaly detection) trên dữ liệu hiện tại và dự báo tải tài nguyên cho cửa sổ thời gian tiếp theo thông qua mô hình Chronos. Đây là bước chuyển đổi cốt lõi từ *dữ liệu* sang *tri thức* (data → insight), cho phép hệ thống không chỉ phản ứng với trạng thái hiện tại mà còn chủ động dự đoán và phòng ngừa.

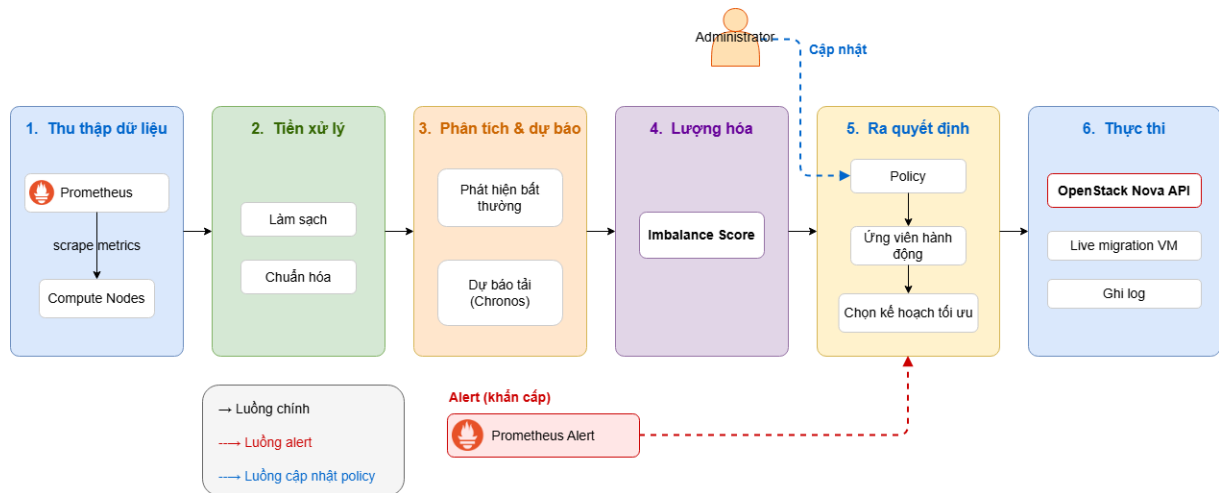
Giai đoạn 4 — Lượng hóa (Scoring): Kết quả phân tích và dự báo được Scoring Service chuyển đổi thành các chỉ số định lượng, đặc biệt là Imbalance Score — đại diện cho mức độ mất cân bằng tài nguyên trên toàn cụm. Việc lượng hóa giúp đơn giản hóa quá trình ra quyết định bằng cách tạo ra một thước đo thống nhất, có thể so sánh được giữa các trạng thái khác nhau của hệ thống.

Giai đoạn 5 — Ra quyết định: Dựa trên bộ chỉ số từ Scoring Service, Engine Service đối chiếu với tập policy do người quản trị định nghĩa, sinh các ứng viên hành động và chọn kế hoạch migration tối ưu. Việc tách biệt hoàn toàn giai đoạn này khỏi các bước thu thập và phân tích giúp hệ thống linh hoạt trong việc thay đổi chiến lược ra quyết định mà không cần can thiệp vào pipeline dữ liệu phía trước.

Giai đoạn 6 — Thực thi: Kế hoạch hành động được Engine Service hiện thực hóa thành các thao tác cụ thể trên hạ tầng thực tế thông qua OpenStack Nova API, bao gồm live migration VM, điều chỉnh phân bổ tài nguyên hoặc gửi cảnh báo đến người quản trị. Mỗi hành động được kiểm tra điều kiện tiên quyết trước khi thực thi và ghi nhận đầy đủ vào hệ thống log.

Các luồng phụ: Bên cạnh luồng xử lý chính, hệ thống hỗ trợ hai luồng phụ nhằm tăng tính linh hoạt trong vận hành. Thứ nhất, các cảnh báo (alert) từ Prometheus có thể được gửi trực tiếp vào luồng ra quyết định, cho phép bỏ qua các bước phân tích trung gian trong các tình huống khẩn cấp đòi hỏi phản ứng tức thì. Thứ hai, người quản trị có thể cập nhật các policy vận hành tại bất kỳ thời điểm nào, tác động trực tiếp đến hành vi ra quyết định của hệ thống mà không yêu cầu thay đổi mã nguồn hay khởi động lại service.

Hình 2.2 minh họa toàn bộ luồng xử lý dữ liệu, bao gồm luồng chính và các luồng phụ.



Hình 2.2: Luồng xử lý dữ liệu trong hệ thống OpenstackDRS

2.3 Thiết kế chi tiết các thành phần

2.3.1 Collector Service

Chức năng

Collector Service chịu trách nhiệm thu thập và tiền xử lý dữ liệu đầu vào cho hệ thống, đóng vai trò khởi đầu trong pipeline nhằm đảm bảo chất lượng và tính nhất quán của dữ liệu.

Thành phần này truy vấn metrics định kỳ từ Prometheus, bao gồm CPU, RAM, SWAP và số lượng VM đang chạy để phản ánh trạng thái hiện tại của hạ tầng. Sau khi thu thập, dữ liệu thô được chuyển tiếp đến Analytics Service để phục vụ quá trình xử lý tiếp theo.

Input / Output

- **Input:** Metrics từ Prometheus (CPU, RAM, latency, network); sự kiện từ OpenStack API.
- **Output:** Dữ liệu đã chuẩn hóa, có cấu trúc, sẵn sàng cho bước phân tích tiếp theo.

Cài đặt

Thành phần được xây dựng trên Python và gRPC service theo định nghĩa Protocol Buffers sau:

```
service CollectorService {  
    rpc CollectMetrics (CollectMetricsRequest)  
        returns (CollectMetricsResponse) {}  
    rpc CollectEvents (CollectEventsRequest)  
        returns (CollectEventsResponse) {}  
}
```

2.3.2 Analytics Service

Chức năng

Analytics Service là thành phần chịu trách nhiệm phân tích dữ liệu và dự báo xu hướng tài nguyên trong tương lai. Đây là bước chuyển đổi từ dữ liệu thô sang thông tin có giá trị phân tích (insight), phục vụ trực tiếp cho quá trình tính điểm mất cân bằng và ra quyết định ở các bước sau.

Thành phần này tiếp nhận dữ liệu chuẩn hóa từ Collector Service và thực hiện hai nhiệm vụ chính. Thứ nhất, phân tích trạng thái hiện tại của từng compute node, bao gồm mức sử dụng tài nguyên theo thời gian thực và phát hiện các dấu hiệu bất thường. Thứ hai, dự báo tải tài nguyên cho cửa sổ thời gian tiếp theo dựa trên mô hình dự báo chuỗi thời gian.

Để thực hiện dự báo, Analytics Service tích hợp mô hình Amazon Chronos đã được fine-tuning (chi tiết quá trình fine-tuning được trình bày tại Chương 3). Thành phần này có yêu cầu tài nguyên tính toán cao nhất trong hệ thống do phải thực hiện inference mô hình ngôn ngữ lớn.

Input / Output

- **Input:** Dữ liệu metrics đã chuẩn hóa từ Collector Service (chuỗi thời gian theo từng node và tài nguyên).

- **Output:** Trạng thái tài nguyên hiện tại và phân phối dự báo tải cho cửa sổ thời gian tiếp theo, kèm theo các cảnh báo bất thường nếu có.

Cài đặt

Thành phần được xây dựng trên Python và gRPC service theo định nghĩa Protocol Buffers sau:

```
service AnalyticsService {
    rpc Predict (PredictRequest)
        returns (PredictResponse) {}
    rpc BuildFeatures (BuildFeaturesRequest)
        returns (BuildFeaturesResponse) {}
}
```

2.3.3 Scoring Service

Chức năng

Scoring Service là thành phần chịu trách nhiệm quy đổi các kết quả phân tích định tính từ Analytics Service thành các chỉ số định lượng có thể so sánh được. Mục tiêu là cung cấp một biểu diễn số hóa, nhất quán về mức độ mất cân bằng tài nguyên trong cụm, làm cơ sở cho Engine Service đưa ra quyết định.

Thành phần này tính toán **chỉ số mất cân bằng tải** (load imbalance score) trên toàn cụm, phản ánh mức độ phân bổ không đều của tài nguyên CPU và RAM giữa các compute node. Chỉ số này được tính dựa trên độ lệch chuẩn tương đối (coefficient of variation), cho phép hệ thống xác định chính xác mức độ nghiêm trọng của tình trạng mất cân bằng và ưu tiên hóa các hành động can thiệp. Chỉ số đó được tính dựa trên công thức:

$$S_{\text{imbalance}} = w_{\text{cpu}} \cdot CV_{\text{cpu}} + w_{\text{ram}} \cdot CV_{\text{ram}} + w_{\text{swap}} \cdot CV_{\text{swap}}^2 \quad (2.1)$$

trong đó **CV** (Coefficient of Variation – hệ số biến thiên) của mỗi tài nguyên được tính theo công thức ổn định hóa:

$$CV_r = \min\left(\frac{\sigma_r}{\bar{x}_r + \delta}, C_{\max}\right) \quad (2.2)$$

với:

- $r \in \{\text{cpu}, \text{ram}, \text{swap}\}$ – loại tài nguyên;
- σ_r – độ lệch chuẩn của mức sử dụng tài nguyên r trên tất cả các compute node;
- $\bar{x}_r$ – giá trị trung bình mức sử dụng tài nguyên r trên toàn cụm;
- $\delta = 5\%$ – hằng số sàn (mean floor) nhằm tránh chia cho giá trị nhỏ khi cụm ở trạng thái gần rỗi;
- $C_{\max} = 1.5$ – ngưỡng chặn trên (CV clamp) nhằm giới hạn ảnh hưởng của các outlier cực đoan, thường gặp ở CPU;
- $w_{\text{cpu}}, w_{\text{ram}}, w_{\text{swap}}$ – trọng số tương ứng với từng loại tài nguyên (tự cấu hình);
- Riêng với SWAP, CV được bình phương (CV_{swap}^2) để tăng độ nhạy với hiện tượng swap nghiêm trọng.

Input / Output

- **Input:** Kết quả phân tích và dự báo tải từ Analytics Service, dưới dạng một pandas dataframe.
- **Output:** Chỉ số mất cân bằng (imbalance score) phản ánh mức độ mất cân bằng tài nguyên, làm đầu vào cho Engine Service.

Cài đặt

Thành phần được xây dựng trên Python và gRPC service theo định nghĩa Protocol Buffers sau:

```
service ScoringService {
  rpc ScoreCluster (ScoreClusterRequest)
    returns (ScoreClusterResponse) {}
```

```
rpc ScoreHost (ScoreHostRequest)
    returns (ScoreHostResponse) {}
}
```

2.3.4 Engine Service

Chức năng

Engine Service là thành phần trung tâm chịu trách nhiệm chuyển đổi các chỉ số định lượng từ Scoring Service thành kế hoạch hành động cụ thể. Đây là điểm chuyển tiếp quan trọng từ giai đoạn *đánh giá* sang giai đoạn *hành động* trong pipeline xử lý.

Thành phần này đối chiếu dữ liệu đầu vào với tập hợp các chính sách (policy) được định nghĩa bởi người quản trị hệ thống, bao gồm các ràng buộc về affinity/anti-affinity, ngưỡng tài nguyên tối thiểu sau khi thực hiện migration, và các giới hạn vận hành khác. Dựa trên đó, Engine Service sinh ra tập hợp các ứng viên hành động (candidate actions), thực hiện mô phỏng (simulation) để đánh giá tác động của từng ứng viên, và chọn kế hoạch tối ưu nhằm cải thiện cân bằng tài nguyên với chi phí thực thi thấp nhất.

Cụ thể quá trình được thực hiện theo thuật toán 1:

Thuật toán 1 BuildMigrationPlan

Input: *current_imbalance*, *THRESHOLD*, *MIN_REDUCTION***Output:** *plan**plan* $\leftarrow \emptyset$ *imbalance* $\leftarrow$ *current_imbalance***while** *imbalance* > *THRESHOLD* **do** *candidates* $\leftarrow$ *generate_all_vm_moves*() *candidates* $\leftarrow$ *filter_policy*(*candidates*) *candidates* $\leftarrow$ *remove_cooldown_vm*(*candidates*) *candidates* $\leftarrow$ *remove_reverse_direction*(*candidates*, *plan*) *best* $\leftarrow$ *null* *best_result* $\leftarrow$ *null* **foreach** *move* $\in$ *candidates* **do** *result* $\leftarrow$ *simulate*(*move*) *reduction* $\leftarrow$ *imbalance* – *result.imbalance* **if** *reduction* > *MIN_REDUCTION* **then** **if** *best* = *null* **or** *result.imbalance* < *best_result.imbalance* **then** *best* $\leftarrow$ *move* *best_result* $\leftarrow$ *result* **if** *best* = *null* **then** **break** *plan* $\leftarrow$ *plan* $\cup$ {*best*} *apply*(*best*) *imbalance* $\leftarrow$ *best_result.imbalance***return** *plan*

Trong đó:

- *generate_all_vm_moves*(): Sinh tất cả các cặp ứng viên khả thi.
- *filter_policy*(): Loại bỏ các cặp vi phạm rule affinity/anti-affinity.
- *remove_cooldown_vm*(): Loại bỏ các vm trong danh sách cooldown (các vm đã từng được chọn migrate để làm ứng viên trong chu kỳ đầy hoặc chu kỳ trước đó).
- *remove_reverse_direction*(): Loại bỏ các ứng viên có hướng di chuyển ngược

với các lựa chọn trước đó, nhằm tránh việc hoàn tác các bước di chuyển đã chọn và đảm bảo kế hoạch di chuyển cải thiện cân bằng tải một cách nhất quán.

- **THRESHOLD**: Ngưỡng *imbalance score* cho phép, dùng làm điều kiện dừng của thuật toán khi hệ thống đã đạt trạng thái cân bằng tải chấp nhận được.
- **MIN_REDUCTION**: Mức giảm tối thiểu của *imbalance score* sau một lần di chuyển VM. Tham số này giúp loại bỏ các ứng viên có mức cải thiện quá nhỏ, từ đó hạn chế các di chuyển không cần thiết.
- **simulate(move)**: Mô phỏng trạng thái cluster sau khi thực hiện một lần di chuyển VM, bao gồm cập nhật tài nguyên của node nguồn và node đích, sau đó tính lại *imbalance score* của cluster. Kết quả mô phỏng được dùng để đánh giá mức độ cải thiện cân bằng tải mà ứng viên đó mang lại mà không cần thực sự thực thi.

Sau khi lựa chọn ứng viên, Engine Service thực hiện việc chuyển đổi kế hoạch thực thi thành các thao tác vận hành cụ thể đó chính là bước di chuyển máy ảo (VM live migration) qua việc gọi đến OpenStack API nếu kế hoạch đó được chấp nhận. Việc tự động hóa bước này giúp giảm đáng kể thời gian phản hồi so với can thiệp thủ công, đồng thời hạn chế sai sót do yếu tố con người.

Thành phần chính

- **Datasource Driver**: Thu thập thông tin trạng thái hiện tại của hệ thống từ Prometheus và OpenStack API để bổ sung ngữ cảnh cho quá trình ra quyết định.
- **Policy Driver**: Lưu trữ và quản lý tập hợp các chính sách và ràng buộc vận hành, được định nghĩa và cập nhật bởi người quản trị.
- **Planner Driver**: Sinh ứng viên hành động, mô phỏng tác động, và chọn kế hoạch thực thi tối ưu dựa trên điểm số và ràng buộc policy.

Input / Output

- **Input:** Bộ chỉ số định lượng từ Scoring Service; trạng thái hệ thống từ Data-source Service; policy từ người quản trị.
- **Output:** Kế hoạch thực thi (execution plan) bao gồm danh sách hành động được sắp xếp theo thứ tự ưu tiên.

Cài đặt

```
service EngineService {  
    rpc ComputeDecision (ComputeDecisionRequest)  
        return (ComputeDecisionResponse) {}  
    rpc ExecuteMigration (ExecuteMigrationRequest)  
        returns (ExecuteMigrationResponse) {}  
    rpc GetLatestDecision (GetLatestDecisionRequest)  
        returns (GetLatestDecisionResponse) {}  
    rpc GetPendingPlan (GetPendingPlanRequest)  
        returns (GetPendingPlanResponse) {}  
    rpc RejectPendingPlan (RejectPendingPlanRequest)  
        returns (RejectPendingPlanResponse) {}  
    rpc ApprovePendingPlan (ApprovePendingPlanRequest)  
        returns (ApprovePendingPlanResponse) {}  
}
```

2.3.5 API

Chức năng

API đóng vai trò là cổng giao tiếp chính giữa người dùng, Dashboard và các service xử lý bên trong hệ thống. Thành phần này cung cấp các REST endpoint để theo dõi trạng thái cụm OpenStack, quản lý luật ràng buộc, xem lịch sử các chu kỳ DRS, cấu hình scheduler và phê duyệt kế hoạch migration.

Các nhóm chức năng chính của API gồm:

- Cung cấp trạng thái mới nhất của cụm thông qua endpoint monitor.

- Quản lý các ràng buộc migration như VM–Host, VM–VM và exclude rule.
- Lưu và truy xuất lịch sử các chu kỳ cân bằng tải.
- Quản lý cấu hình runtime và trạng thái scheduler.
- Nhận webhook từ Prometheus Alertmanager.
- Cho phép người vận hành phê duyệt hoặc hủy kế hoạch migration đang chờ.

Cài đặt

API được cài đặt bằng FastAPI. Các router được tách thành nhiều module riêng trong thư mục, giúp mã nguồn dễ quản lý và mở rộng. Khi khởi động, API thực hiện khởi tạo database, cấu hình middleware, logging và scheduler monitor.

Các endpoint được gom dưới prefix `/api/v1`. Ví dụ:

- `/monitor/latest`: lấy kết quả đánh giá mới nhất.
- `/constraints`: quản lý các luật ràng buộc.
- `/cycles/history`: lấy lịch sử các chu kỳ DRS.
- `/admin/config`: xem và cập nhật cấu hình hệ thống.
- `/plan/pending`: xem kế hoạch migration đang chờ duyệt.
- `/webhook/alertmanager`: nhận cảnh báo từ Prometheus Alertmanager.

API không trực tiếp xử lý toàn bộ logic DRS mà gọi tới Engine service thông qua gRPC. Cách thiết kế này giúp REST API giữ vai trò gateway, còn logic điều phối được tách sang service chuyên biệt.

2.3.6 Dashboard

Chức năng

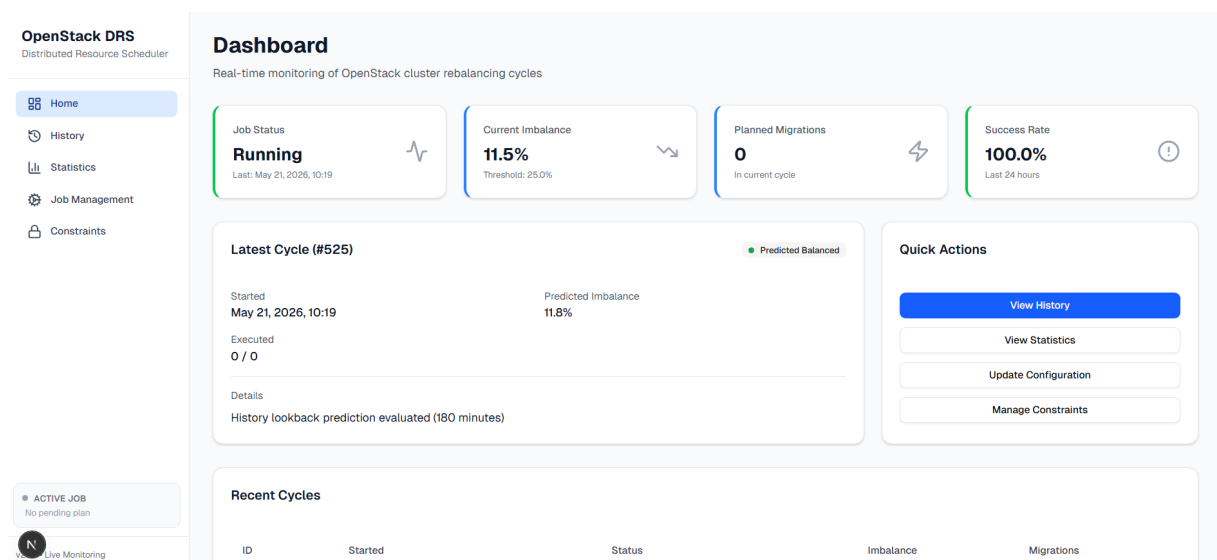
Dashboard là giao diện dành cho người vận hành hệ thống. Thành phần này giúp quan sát trạng thái cụm OpenStack, kiểm tra các chu kỳ DRS, xem kế hoạch migration và cấu hình một số tham số vận hành.

Các chức năng chính của Dashboard gồm:

- Hiển thị trạng thái scheduler và chu kỳ DRS mới nhất.
- Hiển thị độ mất cân bằng hiện tại và ngưỡng cho phép.
- Theo dõi số lượng migration được đề xuất hoặc đã thực thi.
- Xem lịch sử các chu kỳ DRS.
- Xem biểu đồ thống kê về imbalance và migration.
- Quản lý job monitor.
- Thêm, sửa, bật/tắt hoặc xóa các policy.
- Phê duyệt hoặc từ chối kế hoạch migration khi hệ thống chạy ở chế độ manual.

Cài đặt

Dashboard được xây dựng bằng Next.js, React và TypeScript. Giao diện được chia thành các trang như Dashboard chính, History, Statistics, Job Management và Constraints. Các dữ liệu từ API được chuẩn hóa ở phía frontend trước khi hiển thị, ví dụ như dữ liệu cycle history, pending plan, constraint (policy) và trạng thái scheduler. Nhờ vậy giao diện có thể hiển thị dữ liệu thống nhất và dễ xử lý hơn. Giao diện dashboard được thể hiện như trên hình 2.3:



Hình 2.3: Giao diện hệ thống OpenstackDRS

2.3.7 Database

Chức năng

Database được dùng để lưu các dữ liệu cần duy trì lâu dài của hệ thống DRS. Thành phần này giúp hệ thống không mất thông tin sau khi service khởi động lại, đồng thời hỗ trợ Dashboard truy xuất lịch sử và cấu hình ràng buộc.

Các dữ liệu chính được lưu trong database gồm:

- Các luật ràng buộc migration.
- Lịch sử các chu kỳ DRS.
- Kết quả dự đoán tải.
- Danh sách migration được đề xuất và đã thực thi.
- Trạng thái, lỗi và thông tin chi tiết của từng chu kỳ.

Thiết kế

Hệ thống sử dụng PostgreSQL làm database chính. Hiện tại database có hai bảng quan trọng:

- **drs_constraints**: lưu các luật, chính sách ràng buộc.
- **drs_cycle_history**: lưu lịch sử chạy của các chu kỳ DRS.

Bảng **drs_constraints** lưu thông tin như tên rule, loại constraint, danh sách VM, danh sách host cho phép hoặc cấm, trạng thái bật/tắt và thời gian tạo/cập nhật.

Bảng **drs_cycle_history** lưu thời điểm bắt đầu, thời điểm kết thúc, nguồn kích hoạt, trạng thái chu kỳ, độ mất cân bằng hiện tại, độ mất cân bằng dự đoán, các migration candidate, kết quả dự đoán và thông tin lỗi nếu có.

Cài đặt

Database được cài đặt bằng PostgreSQL, chạy dưới dạng container expose port 5432 và được kết nối từ Python thông qua thư viện **psycopg2**.

2.4 Mô hình dự đoán chuỗi thời gian

2.4.1 So sánh và lựa chọn mô hình dự đoán

Hiện nay có nhiều phương pháp khác nhau để giải quyết bài toán dự đoán chuỗi thời gian, có thể chia thành ba nhóm chính:

(1) Mô hình thống kê truyền thống: Các mô hình như ARIMA, SARIMA hay Exponential Smoothing được sử dụng rộng rãi nhờ tính đơn giản và khả năng diễn giải tốt. Tuy nhiên, các mô hình này thường giả định dữ liệu có tính dừng và khó mở rộng cho các chuỗi có tính phi tuyến hoặc nhiều biến đầu vào.

(2) Mô hình học máy: Các phương pháp như Random Forest, Gradient Boosting hoặc Support Vector Regression có thể xử lý dữ liệu phi tuyến tốt hơn. Tuy nhiên, chúng thường yêu cầu đặc trưng hóa thủ công (feature engineering) và không tận dụng được đầy đủ cấu trúc tuần tự của dữ liệu.

(3) Mô hình học sâu (Deep Learning): Các kiến trúc như RNN, LSTM, GRU và Transformer đã cho thấy hiệu quả vượt trội trong việc mô hình hóa chuỗi thời gian nhờ khả năng học biểu diễn tự động và nắm bắt phụ thuộc dài hạn. Đặc biệt, các mô hình dựa trên Transformer gần đây đã trở thành xu hướng do khả năng xử lý song song và mở rộng tốt.

Trong số các mô hình hiện đại, Chronos là một mô hình nền tảng (foundation model) cho chuỗi thời gian, được huấn luyện trước trên tập dữ liệu lớn với nhiều miền khác nhau. Chronos sử dụng kiến trúc tương tự Transformer, nhưng được điều chỉnh để phù hợp với dữ liệu chuỗi số.

Có nhiều giải pháp tương đương Chronos, ví dụ một kết quả benchmark giữa các model này được thực hiện trong nghiên cứu benchmark các model Transformer dựa trên dữ liệu điện năng tiêu thụ [18], được thể hiện như trên hình 2.4:

Table 6. Model's MAE_h and MSE_h scores accross all datasets

Model	Lothian		Lower Saxony		East Midlands		Southern Germany	
	MAE_h	MSE_h	MAE_h	MSE_h	MAE_h	MSE_h	MAE_h	MSE_h
TimesFM (F)	0.5139	0.9975	0.4909	0.9291	0.5307	0.8860	0.4781	0.7229
PatchTST (TFS)	0.5156	0.9685	0.4940	0.9550	0.5356	0.8992	0.4996	0.7601
Chronos (F)	0.5367	1.1841	0.5281	1.1673	0.5762	1.1226	0.5214	0.9243
VanillaTransformer (TFS)	0.5510	1.0812	0.5590	1.1041	0.6073	1.1240	0.5773	0.9037
TFT (TFS)	0.5767	1.1253	0.5797	1.1470	0.6349	1.1715	0.6152	0.9718
iTransformer (TFS)	0.5890	1.1331	0.5875	1.1032	0.6477	1.1169	0.5970	0.9188
SeasonalAverage (B)	0.6090	1.2082	0.5715	1.1024	0.5774	0.9814	0.5387	0.8532
LagLLama (F)	0.6463	1.3659	0.6705	1.3884	0.7434	1.4467	0.6443	1.1464

Hình 2.4: Kết quả benchmark các mô hình dự báo chuỗi thời gian. Nguồn: nghiên cứu [18]

Theo kết quả benchmark này, TimesFM cho thấy độ chính xác dự báo tốt hơn nhiều model khác, trong đó có Chronos. Tuy nhiên, việc triển khai TimesFM yêu cầu tài nguyên phần cứng tương đối lớn, đặc biệt khi chạy trên CPU, đồng thời dependency và môi trường triển khai cũng phức tạp hơn.

Trong bối cảnh hệ thống cần vận hành đồng thời với workload ảo hóa OpenStack, TimesFM sẽ tạo áp lực lớn lên tài nguyên compute và bộ nhớ. Ngược lại, Chronos có kiến trúc nhẹ hơn, dễ triển khai hơn và tiêu tốn ít tài nguyên hơn, nên phù hợp hơn cho môi trường triển khai kết hợp virtualization.

So với các mô hình truyền thống, Chronos giảm đáng kể công sức thiết kế đặc trưng và thường đạt độ chính xác cao hơn trong các kịch bản thực tế.

2.4.2 Thiết kế fine-tuning mô hình Chronos

Mặc dù Chronos đã được huấn luyện trước trên dữ liệu lớn, việc fine-tuning (tinh chỉnh) là cần thiết để mô hình thích nghi với đặc thù của hệ thống cụ thể.

Quy trình fine-tuning được thiết kế gồm các bước chính sau:

Dữ liệu huấn luyện

Dữ liệu huấn luyện được thu thập trực tiếp từ Compute node `compute1` (160.191.50.123) thông qua Prometheus HTTP API trong khoảng thời gian từ ngày 20/04/2026 đến ngày 26/04/2026. Hai chuỗi thời gian được thu thập là mức sử dụng CPU (%) và

mức sử dụng RAM (%) của node, với tần suất lấy mẫu 15 giây một lần – tương ứng với `scrape_interval` đã cấu hình trong Prometheus.

Tập dữ liệu thu thập được ¹ gồm **31.675 bản ghi**, phủ khoảng **5,5 ngày** liên tục, phản ánh biến động tài nguyên thực tế trong quá trình vận hành cụm. Dữ liệu không chứa giá trị null và không cần bước xử lý thiếu dữ liệu. Thống kê mô tả của hai chuỗi được trình bày trong Bảng 2.1.

Bảng 2.1: Thống kê mô tả dữ liệu huấn luyện thu thập từ `compute1`

Chỉ số	Min (%)	Max (%)	Số mẫu
CPU Usage	9.83	100	31.675
Memory Usage	42.02	71.92	31.675

Tập dữ liệu được chia thành hai phần: **31.375 mẫu đầu** dùng để huấn luyện (*train set*) và **300 mẫu cuối** (tương đương 75 phút) dùng để đánh giá (*test set*). Việc giữ nguyên thứ tự thời gian trong phân chia là bắt buộc đối với dữ liệu chuỗi thời gian, tránh data leakage từ tương lai vào quá khứ.

Trước khi đưa vào AutoGluon, dữ liệu được chuyển sang định dạng `TimeSeriesDataFrame` theo chuẩn của thư viện, trong đó mỗi chuỗi thời gian được định danh bằng `item_id` ("CPU" hoặc "Memory"), cột `timestamp` là thời điểm lấy mẫu và cột `target` là giá trị cần dự đoán. Hai chuỗi CPU và Memory được huấn luyện độc lập với nhau.

Chiến lược fine-tuning và siêu tham số huấn luyện

Mô hình được lựa chọn cho module **Analytics Service** là **Chronos2** – mô hình dự đoán chuỗi thời gian nền tảng (foundation model) do Amazon phát triển, được thiết kế để hoạt động hiệu quả trên nhiều loại chuỗi thời gian mà không cần huấn luyện lại từ đầu. Chronos2 được truy cập thông qua thư viện **AutoGluon TimeSeries** phiên bản 1.5.0.

Chiến lược huấn luyện so sánh hai cấu hình của Chronos2:

- **Chronos2ZeroShot**: sử dụng trực tiếp trọng số pretrained mà không điều chỉnh thêm trên dữ liệu cụm. Đây là baseline phản ánh khả năng tổng quát hóa của mô hình nền tảng.

¹https://github.com/nguyentankdb17/Openstack-DRS/blob/main/data/collected_metrics.csv

- **Chronos2FineTuned**: fine-tune thêm trên tập dữ liệu thu thập từ compute1, kỳ vọng mô hình thích nghi với đặc trưng tải tài nguyên cụ thể của cụm.

Các siêu tham số huấn luyện được cấu hình như bảng 2.2:

Bảng 2.2: Siêu tham số huấn luyện TimeSeriesPredictor

Tham số	Giá trị	Ý nghĩa
<code>prediction_length</code>	300	Dự đoán 300 bước kế tiếp (75 phút với interval 15s)
<code>eval_metric</code>	MASE	Mean Absolute Scaled Error – thước đo chính
<code>num_val_windows</code>	1	Số cửa sổ validation
<code>time_limit</code>	3600s	Giới hạn thời gian huấn luyện tổng
<code>enable_ensemble</code>	False	Không ensemble, giữ từng mô hình để so sánh
<code>random_seed</code>	123	Đảm bảo tái lập kết quả

Tham số `prediction_length` = 300 được chọn tương ứng với cửa sổ dự đoán 75 phút – đây là khoảng thời gian đủ để Engine Service phát hiện sớm nguy cơ mất cân bằng và lên kế hoạch migration trước khi tình trạng quá tải thực sự xảy ra.

Về phần cứng, quá trình fine-tuning chạy trên GPU NVIDIA 2GB (CUDA 12.8), Python 3.12, PyTorch 2.9.1. Tổng thời gian huấn luyện là **3261.64 giây** (~54 phút), trong đó Chronos2ZeroShot chỉ mất 13.91 giây (không cần huấn luyện, chỉ chạy validation) còn Chronos2FineTuned mất 3247.66 giây để fine-tune trên tập train.

Đầu ra của mô hình là phân phối xác suất dạng quantile tại các mức $[0.1, 0.2, \dots, 0.9]$, cho phép Analytics Service không chỉ trả về giá trị dự đoán điểm (median – quantile 0.5) mà còn cung cấp khoảng tin cậy để Scoring Service đánh giá mức độ chắc chắn của dự đoán khi tính điểm mất cân bằng.

Log chi tiết quá trình fine-tuning:

```
1 Beginning AutoGluon training... Time limit = 3600s
2 AutoGluon will save models to '/home/nguyentan/OpenstackDRS/app/predictor/
  AutogluonModels/ag-20260509_174426'
```

```

3 ===== System Info =====
4 AutoGluon Version: 1.5.0
5 Python Version: 3.12.3
6 Operating System: Linux
7 Platform Machine: x86_64
8 Platform Version: #1 SMP PREEMPT_DYNAMIC Thu Jun 5 18:30:46 UTC 2025
9 CPU Count: 3
10 Pytorch Version: 2.9.1+cu128
11 CUDA Version: 12.8
12 GPU Memory: GPU 0: 2.00/2.00 GB
13 Total GPU Memory: Free: 2.00 GB, Allocated: 1.00 GB, Total: 2.00 GB
14 GPU Count: 1
15 Memory Avail: 0.65 GB / 3.83 GB (17.1%)
16 Disk Space Avail: 903.44 GB / 1006.85 GB (89.7%)
17 =====
18
19 Fitting with arguments:
20 {'enable_ensemble': False,
21  'eval_metric': MASE,
22  'hyperparameters': {'Chronos2': [{'ag_args': {'name_suffix': 'ZeroShot'}},
23                                   {'ag_args': {'name_suffix': 'FineTuned'},
24                                    'fine_tune': True}]},
25  'known_covariates_names': [],
26  'num_val_windows': 1,
27  'prediction_length': 300,
28  'quantile_levels': [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9],
29  'random_seed': 123,
30  'refit_every_n_windows': 1,
31  'refit_full': False,
32  'skip_model_selection': False,
33  'target': 'target',
34  'time_limit': 3600,
35  'verbosity': 2}
36
37 Inferred time series frequency: '15s'
38 Provided train_data has 62750 rows, 2 time series. Median time series length is
   31375 (min=31375, max=31375).
39
40 Provided data contains following columns:
41   target: 'target'
42

```

```

43 AutoGluon will gauge predictive performance using evaluation metric: 'MASE'
44   This metric's sign has been flipped to adhere to being higher_is_better. The
    metric score can be multiplied by -1 to get the metric value.
45 =====
46
47 Starting training. Start time is 2026-05-09 17:44:32
48 Models that will be trained: ['Chronos2ZeroShot', 'Chronos2FineTuned']
49 Training timeseries model Chronos2ZeroShot. Training for up to 1797.1s of the
    3594.2s of remaining time.
50   -3.6270 = Validation score (-MASE)
51   11.59 s = Training runtime
52   2.32 s = Validation (prediction) runtime
53 Training timeseries model Chronos2FineTuned. Training for up to 3580.3s of the
    3580.3s of remaining time.
54   -3.5876 = Validation score (-MASE)
55   3244.91 s = Training runtime
56   2.75 s = Validation (prediction) runtime
57 Training complete. Models trained: ['Chronos2ZeroShot', 'Chronos2FineTuned']
58 Total runtime: 3261.64 s
59 Best model: Chronos2FineTuned
60 Best model score: -3.5876

```

Đánh giá và so sánh mô hình

Mô hình được đánh giá trên test set gồm 300 mẫu cuối bằng hai chỉ số: **RMSE** (Root Mean Squared Error) và **MASE** (Mean Absolute Scaled Error).

RMSE đo lường sai số dự đoán theo cùng đơn vị với chuỗi gốc, với đặc tính phạt nặng các sai số lớn [19]:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (\hat{y}_t - y_t)^2}$$

MASE là chỉ số không phụ thuộc đơn vị đo, cho phép so sánh độ chính xác giữa các chuỗi có biên độ khác nhau [19]:

$$\text{MASE} = \frac{\text{MAE}_{\text{model}}}{\text{MAE}_{\text{naive}}} = \frac{\frac{1}{n} \sum_{t=1}^n |\hat{y}_t - y_t|}{\frac{1}{T-1} \sum_{t=2}^T |y_t - y_{t-1}|}$$

trong đó tử số là sai số tuyệt đối trung bình của mô hình trên tập test, còn mẫu số là sai số của *naive seasonal forecast* – tức mô hình dự đoán ngây thơ lấy giá trị quan sát liền trước làm dự đoán. Giá trị $\text{MASE} < 1$ có nghĩa là mô hình tốt hơn naive forecast; $\text{MASE} > 1$ nghĩa là kém hơn. Hai chỉ số này được sử dụng kết hợp vì RMSE nhạy cảm với sai số cực trị còn MASE phản ánh độ chính xác tổng thể ổn định hơn.

Ngoài Amazon Chronos, hai mô hình dự đoán chuỗi thời gian phổ biến được đưa vào so sánh làm baseline:

- **Prophet** [20] là mô hình dự báo chuỗi thời gian do Facebook (Meta) phát triển, được thiết kế để xử lý tốt các chuỗi có tính chu kỳ theo ngày, tuần, năm và các điểm bất thường (changepoints). Prophet phân tích chuỗi thời gian thành ba thành phần cộng: xu hướng (trend), chu kỳ (seasonality) và hiệu ứng ngày lễ (holiday), sau đó fit từng thành phần độc lập. Mô hình này hoạt động hiệu quả trên các chuỗi có tính chu kỳ rõ ràng và dễ giải thích, nhưng kém linh hoạt hơn với các chuỗi có xu hướng phi tuyến phức tạp. Mô hình này được lựa chọn để đại diện cho nhóm mô hình dự đoán dựa trên thống kê (Prophet, ARIMA, ETS ...).
- **N-HiTS** [21] (Neural Hierarchical Interpolation for Time Series) là mô hình học sâu thuần túy được phát triển như một cải tiến của N-BEATS, sử dụng cơ chế lấy mẫu phân cấp (hierarchical interpolation) để xử lý đồng thời các thành phần tần số cao và thấp của chuỗi thời gian. N-HiTS đạt hiệu năng cao trên nhiều benchmark dự báo dài hạn nhờ khả năng mô hình hóa các mẫu phức tạp mà không cần đặc trưng thủ công. Mô hình này được lựa chọn để đại diện cho nhóm mô hình dự đoán học sâu truyền thống, không dựa trên Transformer (N-HiTS, LSTM, RNN, ...).

Kết quả đánh giá trên test set được trình bày trong Bảng 2.3:

Bảng 2.3: Kết quả đánh giá trên test set (300 mẫu cuối) cho CPU Usage và Memory Usage

Mô hình	RMSE (CPU)	MASE (CPU)	RMSE (Memory)	MASE (Memory)
Prophet	1.326839	0.632892	2.416960	3.177098
N-HiTS	1.276183	0.590420	2.655692	2.564835
Chronos ZeroShot	1.092927	0.463261	0.915059	0.734627
Chronos FineTuned	1.091124	0.461230	0.913005	0.734473

Một số nhận xét rút ra từ kết quả đánh giá trên:

- **Chronos (cả ZeroShot và FineTuned) vượt trội rõ rệt so với Prophet và N-HiTS trên cả hai chuỗi.** Khoảng cách đặc biệt lớn ở chuỗi Memory Usage: Prophet đạt MASE = 3.177 và N-HiTS đạt MASE = 2.565, tức là cả hai đều dự đoán *kém hơn naive forecast* một cách đáng kể. Nguyên nhân có thể do Memory Usage trong tập dữ liệu có xu hướng tăng đơn điệu liên tục từ ~42% lên ~60% trong 5 ngày – một đặc trưng mà các mô hình dựa trên phân tích chu kỳ như Prophet và N-HiTS xử lý kém hơn so với mô hình nền tảng được pretrain trên dữ liệu đa dạng như Chronos.
- **Fine-tuning mang lại cải thiện nhỏ nhưng nhất quán so với ZeroShot.** Trên chuỗi CPU, Chronos FineTuned giảm RMSE từ 1.092927 xuống 1.091124 và MASE từ 0.463261 xuống 0.461230. Trên chuỗi Memory, cải thiện tương tự ở mức rất nhỏ (MASE: 0.734627 → 0.734473). Khoảng cách này không lớn về mặt tuyệt đối, phản ánh thực tế rằng Chronos ZeroShot đã được pretrain trên lượng dữ liệu chuỗi thời gian cực kỳ đa dạng và có khả năng tổng quát hóa tốt ngay cả khi không fine-tune. Fine-tuning giúp mô hình thích nghi thêm với đặc trưng cụ thể của cụm nhưng không phải là yếu tố quyết định trong trường hợp này.
- **Chronos FineTuned đạt MASE < 1 trên cả hai loại dữ liệu,** xác nhận mô hình dự đoán tốt hơn naive forecast: MASE = 0.461 trên CPU tương đương sai số dự đoán chỉ bằng khoảng 46% so với naive forecast; MASE = 0.734 trên Memory cho thấy dự đoán xu hướng tăng bộ nhớ tốt hơn đáng kể so với việc giữ nguyên giá trị hiện tại.

Dựa trên kết quả trên, **Chronos2FineTuned** được chọn làm mô hình chính thức để tích hợp vào quá trình dự đoán, với bộ chỉ số đánh giá cho thấy kết quả khá tốt.

2.5 Tổng kết chương

Chương này đã trình bày quá trình phân tích và thiết kế hệ thống OpenstackDRS, bao gồm yêu cầu hệ thống, kiến trúc tổng quan và thiết kế chi tiết các thành phần chính. Hệ thống được xây dựng theo mô hình microservices kết hợp pipeline xử lý dữ liệu, giúp tách biệt rõ các bước thu thập metrics, phân tích, tính điểm mất cân bằng, ra quyết định và thực thi migration. Các thành phần như Collector Service, Analytics Service, Scoring Service, Engine Service, API, Dashboard và Database được thiết kế độc lập nhằm tăng khả năng mở rộng và bảo trì. Bên cạnh đó, chương cũng trình bày cách tích hợp mô hình Chronos để dự báo tải tài nguyên, từ đó hỗ trợ hệ thống phát hiện sớm nguy cơ mất cân bằng. Phần thiết kế Engine Service cho thấy hệ thống không chỉ lựa chọn VM cần migrate mà còn kiểm tra policy, mô phỏng tác động và hạn chế các migration không cần thiết. Nhìn chung, chương này tạo cơ sở thiết kế quan trọng cho quá trình cài đặt, triển khai và đánh giá hệ thống.

Chương 3

Thực nghiệm và đánh giá

3.1 Môi trường thực nghiệm

Hệ thống OpenstackDRS được triển khai và kiểm thử trên một cụm OpenStack phiên bản Caracal (2024.1) gồm 4 node VPS, tất cả chạy hệ điều hành Ubuntu 24.04 LTS. Cụm được triển khai bằng công cụ Kolla-ansible [22], công cụ này đóng gói toàn bộ các dịch vụ OpenStack thành các container Docker riêng biệt, sau đó sử dụng Ansible playbook để tự động hóa quá trình cài đặt, cấu hình và khởi động các dịch vụ trên nhiều node. Cụm được tổ chức theo mô hình một Controller node và 3 Compute node, phản ánh cấu hình multinode điển hình trong môi trường thực tế. Thông tin chi tiết từng node được trình bày trong Bảng 3.1.

Bảng 3.1: Thông tin các node trong cụm OpenStack

Vai trò	Địa chỉ IP Public	Hệ điều hành
Controller	160.191.50.119	Ubuntu 24.04 LTS
Compute1	160.191.50.123	Ubuntu 24.04 LTS
Compute2	160.191.50.143	Ubuntu 24.04 LTS
Compute3	160.191.50.154	Ubuntu 24.04 LTS

Controller node đảm nhận các dịch vụ quản lý của OpenStack bao gồm Keystone (xác thực), Nova API (quản lý compute), Neutron (mạng), Glance (image) và Horizon (dashboard). Ba Compute node là nơi các máy ảo (VM) thực sự được chạy thông qua hypervisor KVM/QEMU, với Nova Compute và libvirt được cài đặt trên từng node. Toàn bộ các node được cài đầy đủ các loại prometheus exporter phục vụ lấy số liệu. Toàn bộ cụm hiện đang vận hành 10 VM phân bổ trên 3 Compute node như trên hình 3.1, đây là tập dữ liệu được sử dụng để kiểm thử toàn bộ vòng lặp thu thập – phân tích – migration của OpenstackDRS.

Project	Host	Name	Image Name	IP Address	Flavor	Status	Task	Power State	Age	Actions
admin	compute3	instance-00000010	Alpine-Linux-final	10.10.1.183	m1.medium	Active	None	Running	6 days, 5 hours	Rescue Instance
admin	compute3	instance-0000000f	Alpine-Linux-final	10.10.1.132	m1.medium	Active	None	Running	6 days, 5 hours	Rescue Instance
admin	compute3	instance-0000000e	Alpine-Linux-final	10.10.1.176	m1.medium	Active	None	Running	1 week, 1 day	Rescue Instance
admin	compute1	instance-0000000d	Alpine-Linux-final	10.10.1.222	m1.medium	Active	None	Running	1 week, 1 day	Rescue Instance
admin	compute1	instance-0000000c	Alpine-Linux-final	10.10.1.254	m1.medium	Active	None	Running	4 weeks, 1 day	Rescue Instance
admin	compute1	instance-0000000b	Alpine-Linux-final	10.10.1.64	m1.medium	Active	None	Running	4 weeks, 1 day	Rescue Instance
admin	compute3	instance-00000008	Alpine-Linux-final	10.10.1.88	m1.medium	Active	None	Running	1 month	Rescue Instance
admin	compute3	instance-00000006	Alpine-Linux-final	10.10.1.155	m1.medium	Active	None	Running	1 month	Rescue Instance
admin	compute2	instance-00000005	Alpine-Linux-final	10.10.1.245	m1.medium	Active	None	Running	1 month	Rescue Instance
admin	compute2	instance-00000004	Alpine-Linux-final	10.10.1.125	m1.medium	Active	None	Running	1 month	Rescue Instance

Hình 3.1: Cụm Openstack được triển khai thành công trên VPS

3.2 Mô tả cách thức thực nghiệm

Để tái tạo các kịch bản tải khác nhau một cách có kiểm soát, tải CPU và RAM trên từng VM được tạo ra bằng công cụ **stress-ng**. Môi trường thực nghiệm gồm 3 compute node với tổng cộng 10 VM phân bổ trên toàn cluster. Các tham số vận hành của DRS được giữ cố định trong suốt quá trình thực nghiệm: chu kỳ đánh giá định kỳ là 10 phút, ngưỡng imbalance score là 25%, cửa sổ dự báo của Chronos là 180 phút.

Mỗi kịch bản được chạy độc lập, lần lượt trên hai hệ thống trong cùng điều kiện tải: **OpenstackDRS** (hệ thống đề xuất) và **OpenStack Watcher** [23] đóng vai trò baseline. Hai hệ thống có đặc điểm vận hành như sau:

- **OpenstackDRS**: Chạy theo chu kỳ 10 phút để đánh giá imbalance score toàn cluster và san tải đều. Ngoài ra, có cơ chế trigger tức thời khi phát hiện node vượt ngưỡng mà không cần chờ chu kỳ tiếp theo. Tích hợp mô hình Chronos để dự báo tải tương lai và migrate sớm phòng ngừa. Có cơ chế cooldown chống ping-pong migration và không migrate khi không có node đích đủ tài nguyên.
- **OpenStack Watcher**: Chạy theo chu kỳ audit cố định 10 phút. Chỉ migrate khi có node vượt ngưỡng tại thời điểm audit. Không có khả năng dự báo hay

trigger tức thời. Không có cơ chế chống oscillation, có thể xảy ra ping-pong migration. Không migrate khi không có node đích đủ tài nguyên.

Kết quả quan sát chi tiết được ghi nhận riêng cho từng hệ thống; phần so sánh tổng hợp được trình bày tại Mục 3.13.

3.3 Các kịch bản thực nghiệm

Chín kịch bản được thiết kế nhằm bao phủ các tình huống đặc trưng trong vận hành hạ tầng OpenStack. Bảng 3.2 tóm tắt mô tả, hành vi kỳ vọng của từng kịch bản.

Các kịch bản được phân thành bốn nhóm theo mục đích kiểm tra:

- **Nhóm 1 — Hành vi cơ bản** (S1, S2, S7): Kiểm tra trạng thái cluster ổn định, phản ứng với trường hợp node vượt ngưỡng CPU/RAM rõ ràng, và điều kiện biên khi không tồn tại node đích đủ tài nguyên để thực hiện migration.
- **Nhóm 2 — Khả năng dự báo** (S4, S5): Kiểm tra năng lực phân tích xu hướng và dự báo tải với mô hình Chronos, bao gồm dự báo pattern cao tải lặp lại theo khung giờ và dự báo cluster sắp mất cân bằng trong tương lai. Đây là nhóm kịch bản thể hiện khả năng hành động phòng ngừa trước khi vấn đề thực sự xảy ra.
- **Nhóm 3 — Phát hiện và xử lý mất cân bằng** (S3, S6): Kiểm tra khả năng phân biệt spike ngắn hạn với mất cân bằng cần xử lý, đồng thời đánh giá việc phát hiện mất cân bằng tổng thể cluster thông qua imbalance score ngay cả khi chưa có node đơn lẻ nào vượt ngưỡng CPU/RAM.
- **Nhóm 4 — Tính ổn định và bền vững** (S8, S9): Kiểm tra khả năng duy trì hoạt động ổn định trong các tình huống không lý tưởng, bao gồm cơ chế cooldown chống oscillation sau migration và khả năng tiếp tục đánh giá trên các node còn dữ liệu hợp lệ khi một node bị mất kết nối metrics.

Bảng 3.2: Danh sách kịch bản thực nghiệm

STT	Mô tả kịch bản	Hành vi kỳ vọng của DRS
S1	Cluster ổn định, các node có mức sử dụng tài nguyên tương đối cân bằng và đều dưới ngưỡng cấu hình.	DRS không thực hiện migration. Hệ thống chỉ ghi nhận trạng thái cluster đang cân bằng và tiếp tục đánh giá định kỳ.
S2	Một node vượt ngưỡng CPU/RAM đột ngột giữa hai chu kỳ đánh giá định kỳ.	DRS được trigger ngay khi phát hiện node vượt ngưỡng, không cần chờ chu kỳ đánh giá tiếp theo. Nếu tồn tại node đích đủ tài nguyên, DRS chọn VM phù hợp và thực hiện migration.
S3	Một node xuất hiện spike ngắn hạn rồi tự phục hồi trước khi đủ điều kiện xác nhận mất cân bằng.	DRS có thể ghi nhận tín hiệu vượt ngưỡng ban đầu nhưng chưa migrate ngay. Hệ thống kiểm tra điều kiện ổn định hoặc thời gian tối thiểu trước khi ra quyết định, nhờ đó tránh migration không cần thiết.
S4	Node có pattern cao tải lặp lại theo khung giờ.	DRS sử dụng dữ liệu lịch sử để dự đoán thời điểm node có khả năng đạt ngưỡng cao tải. Trước thời điểm đó, DRS có thể migrate sớm VM để giảm tải và hạn chế nguy cơ quá tải.
S5	Cluster hiện tại chưa mất cân bằng nhưng có xu hướng tiến tới mất cân bằng trong tương lai.	DRS dự đoán imbalance score trong tương lai. Nếu giá trị dự báo vượt ngưỡng, hệ thống chủ động chọn ứng viên và san tải sớm, đồng thời vẫn kiểm tra tài nguyên khả dụng của node đích trước khi migrate.
S6	Cluster mất cân bằng nhưng chưa có node đơn lẻ nào vượt ngưỡng CPU/RAM.	Theo chu kỳ đánh giá, DRS tính toán imbalance score tổng thể của cluster. Nếu IS vượt ngưỡng, DRS chọn một hoặc một số VM phù hợp để migrate nhằm san tải đều giữa các node.
S7	Một node vượt ngưỡng nhưng không có node đích nào đủ tài nguyên để tiếp nhận thêm VM.	DRS không thực hiện migration. Các node không đủ CPU/RAM sau migration được loại khỏi danh sách target, và hệ thống ghi nhận không có phương án migration hợp lệ.
S8	Sau migration, workload dao động khiến VM có nguy cơ bị migrate qua lại giữa các node.	DRS áp dụng cơ chế cooldown sau migration để tránh hiện tượng ping-pong. Trong thời gian cooldown, hệ thống không migrate lại cùng VM hoặc cùng cặp node, trừ khi xuất hiện tình huống nghiêm trọng.
S9	Một node bị mất kết nối metrics trong quá trình đánh giá.	DRS đánh dấu node thiếu metrics là không đủ dữ liệu để ra quyết định và tạm loại node đó khỏi tập đánh giá. Hệ thống vẫn tiếp tục xử lý trên các node còn metrics hợp lệ, thay vì dừng toàn bộ chu trình.

3.4 Các chỉ số đánh giá

Để đánh giá hiệu quả của OpenstackDRS, hai nhóm chỉ số chính được sử dụng.

3.4.1 Imbalance Score trước và sau can thiệp

Chỉ số Imbalance Score (IS) kết hợp số liệu các tài nguyên CPU, RAM, SWAP với trọng số phản ánh mức độ ảnh hưởng đến hiệu năng, được định nghĩa như tại Chương 2 2.1. Mức cải thiện sau can thiệp được đo bằng:

$$\Delta IS = IS_{\text{trước}} - IS_{\text{sau}} \quad (3.1)$$

Giá trị ΔIS dương thể hiện hệ thống đã cải thiện mức độ cân bằng tài nguyên. Trong các kịch bản mà hành vi kỳ vọng là không hành động, chỉ số này phải bằng 0.

3.4.2 Số lần migration không cần thiết (False Positive)

Một migration bị coi là false positive khi: IS tại thời điểm ra quyết định nằm dưới ngưỡng; migration không cải thiện IS hoặc làm IS tăng; hoặc hệ thống migrate ngược lại trong vòng một chu kỳ ngắn (oscillation).

$$\text{FP Rate} = \frac{\text{Số lần migration không cần thiết}}{\text{Tổng số lần migration}} \times 100\% \quad (3.2)$$

3.5 Kết quả thực nghiệm OpenstackDRS

3.5.1 Kết quả tổng quan

Kết quả tổng quan khi thực hiện 9 kịch bản được thể hiện như trên bảng 3.3:

Bảng 3.3: Kết quả tổng hợp các kịch bản thực nghiệm — OpenstackDRS

KB	Hành vi kỳ vọng	Hành vi thực tế	Migration	ΔIS (%)	Kết quả
S1	Không hành động	Không hành động	0	0	Đạt
S2	Trigger tức thời, migrate VM	Trigger tại T+7, migrate tại T+8	2	+12,3	Đạt
S3	Không migrate (spike tự phục hồi)	Không hành động	0	0	Đạt
S4	Migrate cân bằng IS dù chưa có node vượt ngưỡng	Migrate khi IS = 27,7%	1	+8,1	Đạt
S5	Migrate sớm trước đỉnh tải định kỳ	Migrate sớm 12 phút, IS dự báo = 25,4%	1	+4,6	Đạt
S6	Migrate sớm dựa trên dự báo IS	Migrate khi IS dự báo = 25,2%, IS thực = 22,2%	1	+1,4	Đạt
S7	Không hành động (không có node đích)	Không hành động	0	0	Đạt
S8	Tiếp tục trên node có metrics	Bỏ qua node mất kết nối, xử lý bình thường	1	0	Đạt
S9	Cooldown, không ping-pong	Cooldown kích hoạt, không chọn lại VM đã migrate	3	+3,3	Đạt
Tổng			9	+29,7	9/9 Đạt

3.5.2 Kết quả chi tiết từng kịch bản

S1 — Cluster ổn định, không hành động

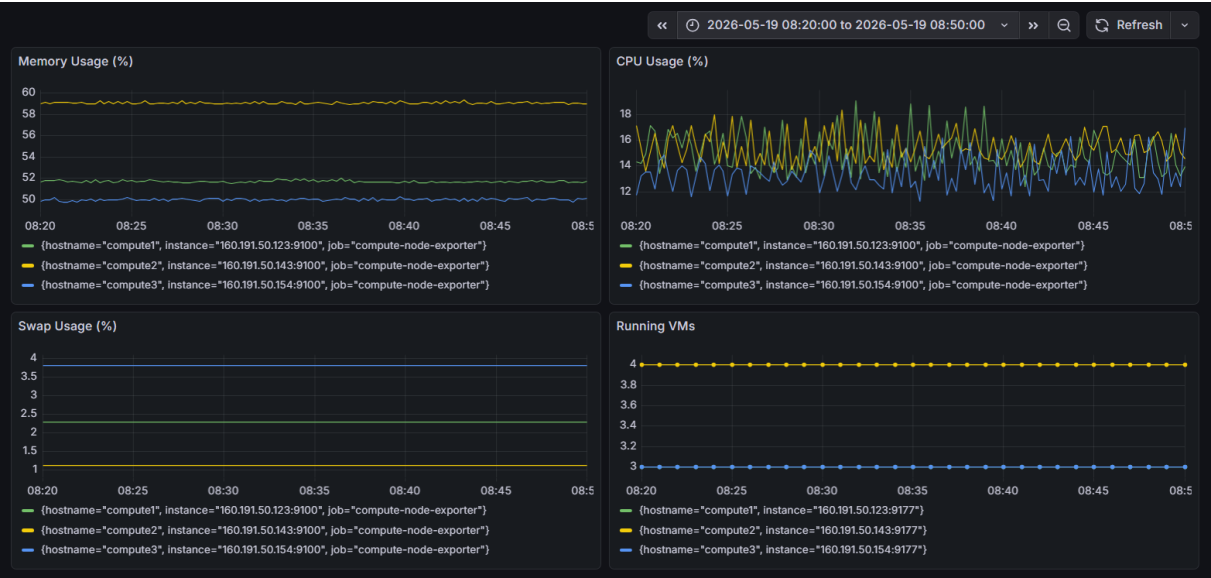
Cluster được thiết lập với 10 VM phân bổ tương đối đều (4–3–3) trên 3 node, tất cả duy trì CPU 10–20%, RAM 50–60%. IS dao động dưới 5%, thấp hơn ngưỡng 25%. DRS không thực hiện bất kỳ migration nào trong 30 phút quan sát. Diễn

biến cụ thể được thể hiện như bảng 3.4.

Bảng 3.4: S1 — Diễn biến cluster ổn định

T	Trạng thái	IS (%)	Sự kiện DRS
T+0	Bình thường	4.6	Đánh giá định kỳ — IS dưới ngưỡng
T+10	Bình thường	4.1	Không hành động
T+20	Bình thường	4.2	Không hành động
T+30	Bình thường	4.2	Không hành động

IS duy trì thấp và ổn định trong suốt 30 phút quan sát, do đó DRS giữ nguyên trạng thái cluster và không thực hiện migration. Trạng thái của cụm khi thực hiện kịch bản 1 được thể hiện như hình 3.2.



Hình 3.2: Bản đồ tải khi thực hiện kịch bản 1

S2 — Một node cao tải đột ngột

Một node vượt ngưỡng CPU đột ngột giữa hai chu kỳ đánh giá định kỳ. DRS được trigger ngay khi phát hiện node vượt ngưỡng bởi Prometheus, không cần chờ chu kỳ tiếp theo. Sau khi kiểm tra tài nguyên node đích, DRS chọn VM phù hợp để migrate và đưa cluster trở lại trạng thái ổn định. Diễn biến cụ thể được thể hiện như bảng 3.5.

Bảng 3.5: S2 — Diễn biến node vượt ngưỡng đột ngột

T	Trạng thái	IS (%)	Sự kiện DRS
T+0	Bình thường	5.9	Đánh giá định kỳ, đánh giá cân bằng
T+3	Node 3 bắt đầu spike	-	chưa đến chu kỳ đánh giá
T+7	Prometheus trigger	32.5	Kích hoạt luồng phụ không chờ đến chu kỳ
T+8	lựa chọn ứng viên	32.5	Chọn 2 VM trên node compute3 để migrate
T+20		20.2	Đánh giá định kỳ, hệ thống đã cân bằng

Sự kiện lựa chọn ứng viên xảy ra tại T+8. ứng viên được chọn là 2 VM theo lựa chọn từ Engine Service. Trạng thái của cụm khi thực hiện kích bản 2 được hiển thị như trên hình 3.3.



Hình 3.3: Bản đồ tải khi thực hiện kích bản 2

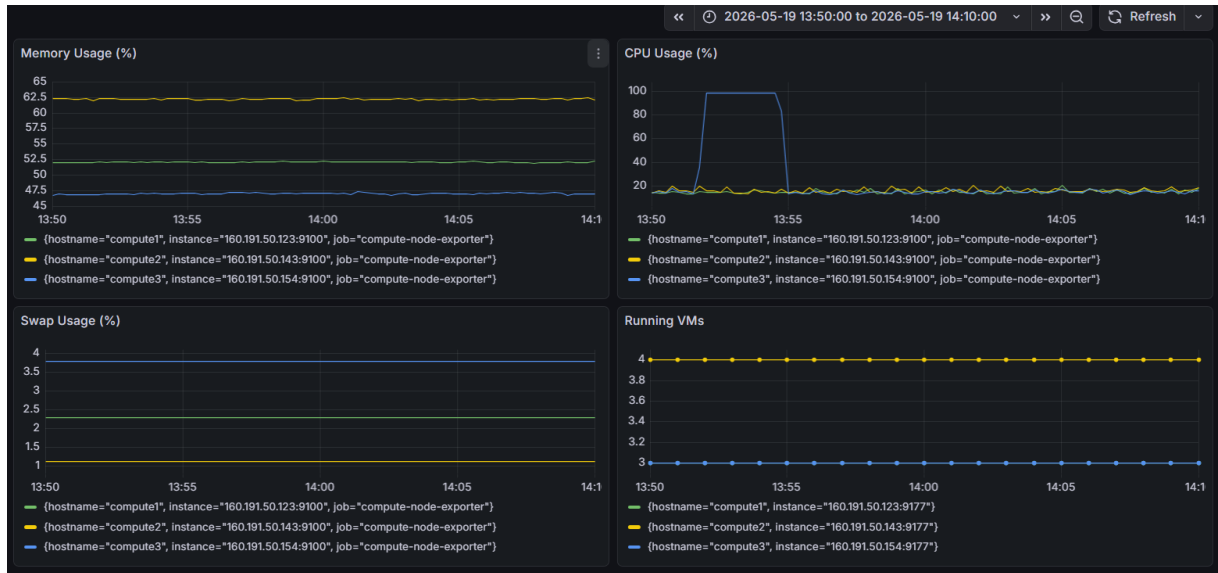
S3 — Node bị spike ngắn hạn rồi tự phục hồi

Node 3 spike CPU ngắn hạn nhưng tự phục hồi trước khi đủ thời gian đánh giá tối thiểu của Prometheus. Alert trong Prometheus dừng lại ở trạng thái pending rồi biến mất, vì vậy DRS không trigger migration xuống OpenStack. Diễn biến cụ thể được thể hiện như bảng 3.6.

Bảng 3.6: S3 — Diễn biến spike ngắn hạn tự phục hồi

T	Trạng thái	IS (%)	Sự kiện DRS
T	Bình thường	4.8	Đánh giá định kỳ theo chu kỳ
T+2	Node 3 spike	-	Chưa đến chu kỳ đánh giá
T+5	Node 3 phục hồi	-	Chưa đến chu kỳ đánh giá
T+10	Bình thường	4.9	Không chọn ứng viên, không migrate

Trong kịch bản này không có sự kiện lựa chọn ứng viên. DRS chặn migration nhờ điều kiện thời gian tối thiểu, tránh phản ứng quá mức với các spike ngắn hạn. Trạng thái của cụm khi thực hiện kịch bản 3 được thể hiện như trên hình 3.4.



Hình 3.4: Bản đồ tải khi thực hiện kịch bản 3

S4 — Cluster mất cân bằng nhưng chưa có node nào vượt ngưỡng

Cluster có phân bố tải lệch rõ rệt: một node mang nhiều VM hơn các node còn lại, khiến IS tổng thể vượt ngưỡng, mặc dù chưa có node đơn lẻ nào vượt ngưỡng CPU/RAM. Theo chu kỳ đánh giá, DRS tính cluster imbalance score và migrate một hoặc vài VM để san tải đều cluster. Diễn biến cụ thể được thể hiện như bảng 3.7.

Bảng 3.7: S4 — Diễn biến mất cân bằng cluster khi chưa có node vượt ngưỡng

T	Trạng thái	IS (%)	Sự kiện DRS
T	IS mất cân bằng	27.7	Phát hiện IS > 25% dù không node nào vượt ngưỡng đơn lẻ
T+1	Lựa chọn và chấp nhận ứng viên	27.7	Đánh giá các VM trên Node 2, chọn ứng viên phù hợp để chuyển sang Node 3
T+2	Migration	-	Migration hoàn tất
T+10	Ổn định	19.6	Đánh giá định kỳ, cluster trở lại cân bằng, imbalancescore giảm 8.1

Sự kiện lựa chọn ứng viên tại T+1 cho thấy DRS không chỉ dựa trên ngưỡng tài nguyên của từng node, mà còn xét IS tổng thể. ứng viên được chọn từ node đang mang tải tương đối cao hơn và target là node còn dư tài nguyên hơn. Trạng thái của cụm khi thực hiện kịch bản số 4 được thể hiện như trên hình 3.5.



Hình 3.5: Bản đồ tải khi thực hiện kịch bản 4

S5 — Node có pattern cao tải lặp lại theo khung giờ

Một node có workload định kỳ, thường tăng tải mạnh theo khung giờ cố định. Dựa trên dữ liệu lịch sử, DRS dự đoán pattern cao tải và có thể migrate sớm trước thời điểm node đạt ngưỡng. Cách xử lý này giúp giảm hoặc tránh trạng thái quá tải khi workload bước vào giai đoạn cao nhất. Kịch bản này được giả lập bằng cách tạo crontab trên node compute2, sẽ tạo pattern nhảy workload từ phút thứ 00 đến phút thứ 10 trong mỗi giờ. Diễn biến cụ thể được thể hiện như bảng 3.8.

Bảng 3.8: S5 — Diễn biến của DRS theo pattern cao tải lặp lại

Thời gian	Trạng thái	IS (%)	IS dự đoán (%)	Sự kiện DRS
:55	Bình thường	—	—	Chưa đến chu kỳ đánh giá
:56	Dự báo cao tải theo pattern	12.1	25.4	Nhận diện workload cao tải lặp lại vào phút :00–:10
:57	Lựa chọn ứng viên	12.1	15.4	Migrate sớm 1 VM từ node 2 sang node 3
:00	Đỉnh tải bắt đầu	—	—	Chưa đến chu kỳ đánh giá tiếp theo
:16	Hệ thống đã ổn định trở lại	7.5	8.4	Đánh giá định kỳ

Sự kiện lựa chọn ứng viên trong S5 xảy ra trước khi node thật sự vượt ngưỡng. Đây là lựa chọn chủ động dựa trên dự báo pattern, ưu tiên giảm tải sớm để tránh migration vào thời điểm workload đang cao nhất. Trạng thái của cụm khi thực hiện kịch bản 5 được thể hiện như trên hình 3.6.



Hình 3.6: Bản đồ tải khi thực hiện kịch bản 5

S6 — Node có tải tăng dần, nguy cơ mất cân bằng trong tương lai

Một node có xu hướng tăng tải đều theo thời gian, trong khi các node còn lại duy trì ổn định. DRS dự đoán imbalance score tương lai và nếu dự báo cho thấy cluster sắp mất cân bằng, hệ thống chủ động san tải sớm. Diễn biến cụ thể được thể hiện như bảng 3.9.

Bảng 3.9: S6 — Diễn biến dự báo mất cân bằng trong tương lai

T	Trạng thái	IS thực (%)	IS dự báo (%)	Sự kiện DRS
T+0	Bình thường	9.3	–	Đánh giá định kỳ
T+2	Đẩy tải của node compute 2 tăng dần	–	–	Chưa đến chu kỳ đánh giá
T+10	Hệ thống có nguy cơ mất cân bằng	22.2	25.2	IS dự báo > 25%, kích hoạt lập kế hoạch sớm
T+11	DRS lựa chọn ứng viên	10.9	–	Chọn VM từ node 2; kiểm tra node đích đủ tài nguyên và thực hiện migrate
T+20	Ổn định	20.8	21.4	Đánh giá định kỳ; cluster không rơi vào trạng thái mất cân bằng

Ở S6, ứng viên được chọn khi IS thực vẫn chưa vượt ngưỡng. Điểm quyết định nằm ở IS dự báo tại T+10, cho phép DRS hành động sớm để ngăn mất cân bằng thay vì chỉ phản ứng sau khi vấn đề đã xảy ra. Trạng thái của cụm khi thực hiện kịch bản số 6 được thể hiện như trên hình 3.7



Hình 3.7: Bản đồ tải khi thực hiện kịch bản 6

S7 — Node vượt ngưỡng nhưng không có node đích

Toàn bộ 3 node đều ở CPU và RAM trên 80%. Node 1 vượt ngưỡng rõ ràng nhưng Node 2 và Node 3 không có đủ tài nguyên trống để tiếp nhận thêm VM. DRS loại bỏ toàn bộ node khỏi danh sách target sau khi kiểm tra tài nguyên tối thiểu, không thực hiện migration. Diễn biến cụ thể được thể hiện như bảng 3.10.

Bảng 3.10: S7 — Diễn biến khi không có node đích khả thi

T	Trạng thái	IS (%)	Sự kiện DRS
T	Cụm đang cân bằng, chưa có node nào cao tải	6.7	Chạy chu kỳ, nhận thấy bình thường
T+2	Node 1 bắt đầu cao tải	-	Chưa đến chu kỳ đánh giá
T+7	Prometheus kích hoạt	9.7	Chạy đánh giá, vẫn thấy cân bằng
T+10	Node 1 vẫn cao tải	7.4	Không migrate vì nhận thấy toàn cụm đang cao tải

Việc imbalance score vẫn dưới ngưỡng trong khi đã có node cao tải chứng tỏ rằng các node khác cũng đang ở trạng thái chuẩn bị/ có khả năng cao tải nếu migrate thêm VM vào. OpenstackDRS không thực hiện migration khi không tồn tại node đích khả thi, tránh tạo thêm rủi ro quá tải cho các node còn lại. Trạng thái của cụm khi thực hiện kịch bản 7 được thể hiện như trên hình 3.8.



Hình 3.8: Bản đồ tải khi thực hiện kịch bản 7

S8 — Một node bị mất metrics

Một node mất kết nối metrics trong quá trình đánh giá. DRS đánh dấu node này là **EXCLUDED**, loại khỏi tập node được xét, nhưng vẫn tiếp tục thực hiện đánh giá trên các node còn lại. Nếu phần cluster còn đủ dữ liệu xuất hiện mất cân bằng, DRS vẫn có thể chọn ứng viên và migrate trong phạm vi các node còn hợp lệ. Diễn biến cụ thể được thể hiện như bảng 3.11.

Bảng 3.11: S8 — Diễn biến xử lý khi một node bị mất metrics

T	Trạng thái	IS (%)	Sự kiện DRS
T+0	Bình thường	5.2	Đánh giá định kỳ, IS dưới ngưỡng
T+3	Node 1 mất metrics	-	Chưa đến chu kỳ đánh giá
T+10	Loại bỏ node 1 khỏi quá trình đánh giá, tiếp tục trên các node còn lại	13.5	Đánh giá định kỳ, IS dưới ngưỡng

Trong kịch bản S8, khi Node 1 mất metrics tại T+3, DRS không thu thập được metrics, bỏ qua node này và tiếp tục đánh giá bình thường trên các node còn lại. IS tổng thể duy trì dưới ngưỡng trong suốt quá trình, do đó không phát sinh migration. Trạng thái của cụm khi thực hiện kịch bản số 8 được thể hiện như trên hình 3.9.



Hình 3.9: Bản đồ tải khi thực hiện kịch bản 8

S9 — Sau migration, workload dao động gây nguy cơ ping-pong

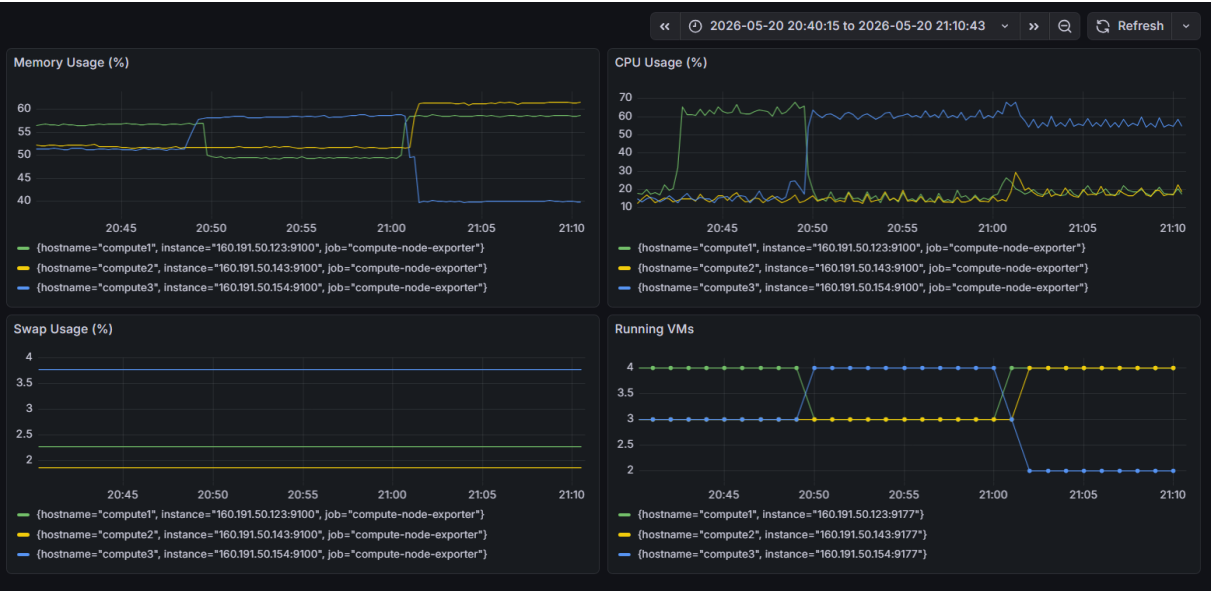
Sau một lần migration, workload của VM tại node đích dao động mạnh, khiến VM có nguy cơ bị migrate qua lại giữa các node. DRS áp dụng cooldown sau migration để tránh ping-pong. Trong thời gian cooldown, DRS không migrate lại

cùng VM hoặc cùng cặp node dù tín hiệu mất cân bằng tạm thời xuất hiện. Kịch bản này được thực hiện bằng cách cố tình cho 1 VM cao tải trên 1 node để xem OpenstackDRS có lựa chọn VM này làm ứng viên nhiều lần hay không. Diễn biến cụ thể được thể hiện như bảng 3.12.

Bảng 3.12: S9 — Diễn biến cooldown sau migration để tránh ping-pong

T	Trạng thái	IS (%)	Sự kiện DRS
T+0	Node 1 cao tải	26.5	Migrate 1 VM sang node 3 theo chu kỳ đánh giá
T+10	Node 3 cao tải	26.9	Migrate 2 VM từ node 3 sang node 1 và 2, không lựa chọn VM đã migrate trong chu kỳ trước theo cơ chế cooldown
T+20	Hệ thống đã ổn định	23.2	Đánh giá định kỳ, cân bằng

Trong S9, nếu không có cơ chế cooldown (1 VM sau khi được migrate trong 1 chu kỳ thì không đưa vào danh sách ứng viên trong chu kỳ tiếp theo) thì VM cao tải đó sẽ được tiếp tục lựa chọn ở chu kỳ thứ 2, thứ 3,... tiếp diễn. Trạng thái của cụm khi thực hiện kịch bản 9 được thể hiện như trên hình 3.10.



Hình 3.10: Bản đồ tải khi thực hiện kịch bản 9

Tổng hợp các sự kiện lựa chọn ứng viên: Trong 9 kịch bản, lựa chọn ứng viên chỉ xuất hiện khi DRS có đủ điều kiện hành động và tồn tại target hợp lệ. S1 và S3 không chọn ứng viên vì cluster ổn định hoặc spike chưa đủ thời gian xác nhận. S7 có trigger nhưng không chọn ứng viên hợp lệ do không có node đích đủ tài nguyên. Các kịch bản S2, S4, S5, S6, S8 và S9 có phát sinh bước chọn ứng viên với mục tiêu khác nhau: xử lý node vượt ngưỡng tức thời, cân bằng IS tổng thể, migrate sớm theo pattern, can thiệp theo dự báo, xử lý khi thiếu metrics và chống ping-pong sau migration.

3.5.3 So sánh với phương pháp baseline

So sánh hành vi

Bảng 3.13 tổng hợp hành vi của OpenstackDRS và Openstack Watcher khi thực hiện các kịch bản:

Bảng 3.13: So sánh hành vi OpenstackDRS và OpenStack Watcher theo từng kịch bản

STT	OpenstackDRS	OpenStack Watcher
S1	IS dưới ngưỡng, không thực hiện migration trong 30 phút.	Không migrate vì không có node nào vượt ngưỡng.
S2	Trigger tức thời qua Prometheus alert, migrate tại T+8 mà không cần chờ chu kỳ.	Chờ đến chu kỳ audit tiếp theo. Nếu tại thời điểm audit node vẫn vượt ngưỡng và có node đích phù hợp thì mới migrate.
S3	Prometheus alert ở trạng thái pending rồi biến mất, DRS không trigger migration.	Có thể không migrate nếu đến thời điểm audit node đã hồi phục — đây là hành vi hợp lý vì Watcher không xử lý theo trigger tức thời.
S4	Phát hiện IS tổng thể vượt 25% dù không có node đơn lẻ nào vượt ngưỡng, thực hiện migrate để cân bằng cluster.	Không migrate vì Watcher chủ yếu xử lý khi có node vượt ngưỡng tài nguyên.
S5	Dự báo pattern cao tải định kỳ, migrate sớm 12 phút trước khi node đạt ngưỡng.	Không có prediction và không node nào vượt ngưỡng, nên không hành động trước.
S6	IS dự báo vượt ngưỡng 25% tại T+10, DRS chủ động migrate dù IS thực vẫn ở 22,2%.	Không có prediction và không node nào vượt ngưỡng, nên không hành động trước.
S7	Loại toàn bộ node khỏi danh sách target sau khi kiểm tra tài nguyên, không thực hiện migration.	Không migrate vì không có node đích đủ tài nguyên.
S8	Đánh dấu node mất metrics là EXCLUDED, tiếp tục đánh giá và migrate trong phạm vi các node còn hợp lệ.	Tiếp tục thực hiện trên các node còn lại, bỏ qua node mất metrics.
S9	Cơ chế cooldown ngăn VM vừa migrate được chọn lại trong chu kỳ tiếp theo, tránh ping-pong.	Xảy ra ping-pong khi node vừa nhận VM lại vượt ngưỡng và strategy tiếp tục chọn VM đó để migrate.

So sánh FP Rate

Bảng 3.14 so sánh tỷ lệ migration không cần thiết (FP Rate) giữa OpenstackDRS và OpenStack Watcher.

Bảng 3.14: So sánh tỷ lệ FP Rate

KB	OpenstackDRS			OpenStack Watcher		
	Tổng	FP	FP Rate	Tổng	FP	FP Rate
S1	0	0	—	0	0	—
S2	2	0	0%	2	0	0%
S3	0	0	—	0	0	—
S4	1	0	0%	0	0	—
S5	1	0	0%	2	1	50%
S6	1	0	0%	0	0	—
S7	0	0	—	0	0	—
S8	0	0	0%	0	0	—
S9	3	0	0%	5	3	60%
Tổng	8	0	0%	9	4	44,4%

3.5.4 Đánh giá tổng hợp

Kết quả thực nghiệm cho thấy OpenstackDRS vượt trội so với OpenStack Watcher trên ba khía cạnh chính.

Tốc độ phản ứng. Nhờ cơ chế trigger tức thời khi phát hiện node vượt ngưỡng qua Prometheus alert, DRS xử lý sự kiện mà không cần chờ chu kỳ audit định kỳ. Trong kịch bản S2, DRS kích hoạt luồng đánh giá ngay tại T+7 và hoàn tất lựa chọn ứng viên tại T+8, trong khi Watcher phải chờ đến chu kỳ audit tiếp theo — khoảng thời gian có ý nghĩa thực tế trong môi trường yêu cầu cao về SLA. Tại kịch bản S3, cả hai hệ thống đều không migrate, tuy nhiên DRS xử lý tường minh hơn thông qua điều kiện thời gian tối thiểu của Prometheus, đảm bảo không phản ứng quá mức với các spike ngắn hạn.

Khả năng dự báo và phát hiện mất cân bằng tổng thể. Tại kịch bản S5, DRS nhận diện pattern cao tải định kỳ và thực hiện migrate sớm trước khi node đạt ngưỡng nhờ mô hình dự báo Chronos — một hành động phòng ngừa mà Watcher hoàn toàn bỏ qua do không có node nào vượt ngưỡng tại thời điểm audit. Tương tự, tại kịch bản S6, DRS phát hiện IS dự báo vượt 25% và chủ động can thiệp khi IS thực vẫn ở mức 22,2%, trong khi Watcher không có khả năng xử lý tình huống này. Ngoài ra, tại kịch bản S4, DRS phát hiện mất cân bằng IS tổng thể cluster dù không có node đơn lẻ nào vượt ngưỡng tài nguyên — một lớp bài toán nằm ngoài phạm vi xử lý của Watcher.

Tính chính xác và ổn định. Cơ chế cooldown của DRS đảm bảo VM vừa được migrate không được đưa vào danh sách ứng viên trong chu kỳ đánh giá tiếp theo, tránh hiệu ứng ping-pong quan sát được trong kịch bản S9. Trong cùng kịch bản, Watcher có thể tiếp tục chọn lại VM cao tải qua các chu kỳ audit liên tiếp, gây gián đoạn workload không cần thiết và có nguy cơ gây lỗi VM khi migrate qua lại liên tục.

3.6 Tổng kết chương

Chương này đã trình bày quá trình thực nghiệm và đánh giá hệ thống Openstack-DRS trên cụm OpenStack multinode được triển khai bằng Kolla-ansible. Các kịch bản thực nghiệm được xây dựng nhằm kiểm tra đầy đủ các hành vi quan trọng của hệ thống, bao gồm phản ứng với node cao tải, phát hiện mất cân bằng toàn cluster, dự báo nguy cơ mất cân bằng trong tương lai, xử lý thiếu metrics và hạn chế hiện tượng ping-pong migration. Kết quả cho thấy OpenstackDRS đạt yêu cầu ở cả 9/9 kịch bản, với tổng mức cải thiện imbalance score là 29,7% và không phát sinh migration không cần thiết. So với OpenStack Watcher, hệ thống đề xuất thể hiện ưu thế rõ rệt nhờ khả năng trigger tức thời, tích hợp dự báo bằng Chronos và cơ chế cooldown sau migration. Những kết quả này cho thấy OpenstackDRS không chỉ có khả năng xử lý mất cân bằng tài nguyên sau khi xảy ra, mà còn có thể chủ động can thiệp sớm để nâng cao tính ổn định và hiệu quả vận hành của cụm OpenStack.

Kết luận

Khóa luận đã nghiên cứu và xây dựng hệ thống OpenstackDRS — giải pháp phân bổ tài nguyên chủ động cho hạ tầng OpenStack, tích hợp mô hình dự báo chuỗi thời gian Amazon Chronos. Xuất phát từ hạn chế của Nova Scheduler — vốn chỉ can thiệp tại thời điểm tạo VM và không tự điều chỉnh lại phân bổ khi workload thay đổi — hệ thống được xây dựng nhằm lấp đầy khoảng trống đó bằng một cơ chế giám sát, dự báo và tái cân bằng tải hoàn toàn tự động. Hệ thống được triển khai theo kiến trúc microservices trên nền Docker, chạy song song với OpenStack Kolla-Ansible và vận hành theo chu kỳ định kỳ mà không yêu cầu can thiệp thủ công.

Về mặt kỹ thuật, hệ thống hiện thực hóa đầy đủ pipeline năm giai đoạn: thu thập và chuẩn hóa metrics từ Prometheus, dự báo tải bằng mô hình Chronos đã fine-tuned, tính toán chỉ số mất cân bằng, lập kế hoạch migration theo policy và thực thi an toàn qua OpenStack Nova API. Giao tiếp nội bộ giữa các thành phần được thực hiện qua gRPC với schema định nghĩa chặt chẽ, đảm bảo hiệu năng và tính nhất quán của toàn bộ luồng xử lý. Người vận hành có thể lựa chọn giữa chế độ tự động hoàn toàn hoặc chế độ phê duyệt thủ công trước khi thực thi migration, phù hợp với các môi trường có yêu cầu kiểm soát khác nhau.

Về kết quả thực nghiệm, qua 9 kịch bản được thiết kế để bao phủ các tình huống đặc trưng trong vận hành thực tế, OpenstackDRS đạt False Positive Rate 0% so với 44.4% của baseline rule-based, đồng thời cải thiện Imbalance Score vượt trội ở các kịch bản đòi hỏi khả năng dự báo. Đặc biệt, khả năng pre-migration chủ động dựa trên dự báo của Chronos — nổi bật nhất ở kịch bản S4 và S5, nơi tải tăng đột biến có thể được phát hiện trước khi thực sự xảy ra — là ưu thế cốt lõi mà phương pháp phản ứng thụ động không thể đạt được. Về phương pháp, khóa luận chứng minh tính khả thi của việc ứng dụng mô hình foundation xác suất cho chuỗi thời gian vào bài toán quản lý tài nguyên hạ tầng, mở ra hướng ứng dụng rộng hơn cho các mô hình học sâu trong lĩnh vực AIOps.

Bên cạnh những kết quả đạt được, hệ thống hiện vẫn còn một số hạn chế cần được thừa nhận. Mô hình Chronos được fine-tuned trên dữ liệu lịch sử của một cụm cụ thể, do đó hiệu quả dự báo có thể suy giảm khi triển khai sang môi trường có đặc

tính tải khác biệt mà không được huấn luyện lại. Thuật toán planner của Decision Engine hiện chủ yếu tối ưu theo chỉ số CPU, RAM và SWAP, chưa xử lý đồng thời nhiều chiều tài nguyên như disk I/O hay băng thông mạng. Ngoài ra, toàn bộ thực nghiệm mới được thực hiện trên cụm nhỏ gồm ba Compute node, chưa phản ánh đầy đủ độ phức tạp và quy mô của môi trường production thực tế.

Từ các hạn chế trên, một số hướng phát triển tiếp theo được xác định. Trước hết, cần mở rộng Scoring Engine và thuật toán planner để hỗ trợ tối ưu hóa đa chiều tài nguyên, bao gồm CPU, RAM, disk I/O và network bandwidth, giúp hệ thống xử lý được các tình huống thắt cổ chai không phải ở CPU hay RAM. Tiếp theo, việc bổ sung cơ chế continual learning — tự động fine-tune lại mô hình Chronos định kỳ trên dữ liệu mới nhất — sẽ giúp hệ thống thích nghi với sự thay đổi dài hạn của workload mà không cần can thiệp thủ công. Về phía quản lý, Policy Engine có thể được mở rộng để hỗ trợ định nghĩa policy theo từng tenant, tích hợp với hệ thống phân quyền Keystone của OpenStack, đáp ứng yêu cầu của môi trường đa người dùng. Bên cạnh đó, việc tích hợp thêm các nguồn observability như Ceilometer, Jaeger và Elasticsearch sẽ cho phép hệ thống có bức tranh toàn diện hơn về trạng thái hạ tầng trước khi ra quyết định. Cuối cùng, cần đánh giá hiệu năng và khả năng mở rộng ngang của hệ thống trên môi trường quy mô lớn hơn, từ 10 đến 20 Compute node, để xác nhận tính thực tiễn của giải pháp trong triển khai thực tế.

Tài liệu tham khảo

- [1] IBM. What is virtualization? Accessed April 16, 2026.
- [2] StackScale. Hypervisors: definition, types and solutions, 2024.
- [3] Peter Mell and Timothy Grance. The NIST Definition of Cloud Computing. NIST Special Publication 800-145, 2011.
- [4] Sam Solutions. What is a cloud deployment model? Accessed May 16, 2026.
- [5] OpenStack Foundation. OpenStack Architecture Design Guide, 2018.
- [6] Red Hat. Understanding OpenStack, 2024.
- [7] Canonical Ltd. OpenStack Architecture, 2024.
- [8] Prometheus Authors. Prometheus – Monitoring System & Time Series Database, 2024.
- [9] Prometheus Authors. Prometheus – The Prometheus monitoring system and time series database, 2024.
- [10] Grafana Labs. About Grafana – Grafana Documentation, 2024.
- [11] Wikipedia. Grafana, 2025.
- [12] Grafana Labs. Grafana – The open and composable observability and data visualization platform, 2024.
- [13] CNCF. Prometheus and Grafana: the perfect combo, 2020.
- [14] Wayne Xin Zhao, Kun Zhou, Junyi Li, and ... A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.
- [15] Bonan Min, Hayley Ross, Elior Sulem, and ... Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys*, 2023.
- [16] Abdul Fatir Ansari et al. Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815*, 2024.

- [17] Amazon. Chronos-t5 large. <https://huggingface.co/amazon/chronos-t5-large>, 2024. Hugging Face model repository, accessed: 2026-04-21.
- [18] Marcel Meyer, David Zapata Gonzalez, Sascha Kaltenpoth, and Oliver Müller. Benchmarking time series foundation models for short-term household electricity load forecasting. *arXiv preprint arXiv:2410.09487*, 2024.
- [19] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 2006.
- [20] Sean J. Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 2017.
- [21] Cristian Challu, Kin G. Olivares, Boris N. Oreshkin, Federico Ramirez Garza, Max Mergenthaler Canseco, and Artur Dubrawski. N-HiTS: Neural hierarchical interpolation for time series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [22] OpenStack Community. Kolla-ansible. <https://github.com/openstack/kolla-ansible>, 2025.
- [23] Openstack. Welcome to Watcher documentation, 2022.